

第四周 三维空间中的旋转与平移

1.1 实验一：三维空间单平移变换与位置状态表述

1.1.1 实验目的

1. 掌握空间平移变换的代数加法本质与位置状态的矩阵表达。
2. 理解平移向量 $\mathbf{t}$ 是描述刚体系统“位置状态”的数学载体。
3. 学会使用 NumPy 进行向量加法与齐次矩阵平移计算。
4. （选做）通过动画模块与飞机模块观察实体模型从点 A 平移至点 B 的线性运动过程。

1.1.2 实验环境

1. 软件环境：Python 3.8+
2. 核心依赖包：numpy、matplotlib
3. 工程辅助模块：animator_3d.py

1.1.3. 实验原理

在固定的三维空间参考坐标系 $\{0\}$ 下，物体的空间位置由其几何中心或特征点的位置向量 $\mathbf{p} = [x, y, z]^T \in \mathbb{R}^3$ 唯一确定。

当物体从初始位置点 A （位置向量为 $\mathbf{p}_A$ ）沿空间指定方向发生平移运动到达目标点 B （位置向量为 $\mathbf{p}_B$ ）时，其运动状态的变化由平移向量（Translation Vector） $\mathbf{t}$ 描述：

$$\mathbf{t} = \begin{bmatrix} t_x \\ t_y \\ t_z \end{bmatrix}$$

其中 t_x, t_y, t_z 分别表示沿世界坐标系 X, Y, Z 三个正交轴向的线性位移分量。

空间平移变换在数学上遵循严格的欧几里得向量加法法则，目标位置点 B 的坐标计算公式为：

$$\mathbf{p}_B = \mathbf{p}_A + \mathbf{t} = \begin{bmatrix} x_A \\ y_A \\ z_A \end{bmatrix} + \begin{bmatrix} t_x \\ t_y \\ t_z \end{bmatrix} = \begin{bmatrix} x_A + t_x \\ y_A + t_y \\ z_A + t_z \end{bmatrix}$$

平移变换仅改变物体在参考系中的位置坐标，不改变物体自身的朝向与姿态。

在 Python 原生列表中，使用 $+$ 运算符会执行列表拼接（如 $[1, 2] + [3, 4]$ 得到 $[1, 2, 3, 4]$ ），而无法完成数学上的向量加法。

而在 NumPy 中，ndarray 数组重载了代数算术运算符 $+$ 。当两个数组的形状一致时，NumPy

会自动执行底层的逐元素并行加法：

$$\mathbf{p}_B = \mathbf{p}_A + \mathbf{t_vec}$$

$\mathbf{p}_B$, $\mathbf{p}_A$, $\mathbf{t_vec}$ 都是 numpy 数组

该代码在底层的数学映射过程为：

$$\mathbf{p}_B = \begin{bmatrix} \mathbf{p}_A[0] + \mathbf{t_vec}[0] \\ \mathbf{p}_A[1] + \mathbf{t_vec}[1] \\ \mathbf{p}_A[2] + \mathbf{t_vec}[2] \end{bmatrix} = \begin{bmatrix} x_A + t_x \\ y_A + t_y \\ z_A + t_z \end{bmatrix}$$

1.1.4 实验要求

1. 设定空间初始点 A 的坐标为 $\mathbf{p}_A = [1.0, 0.5, 0.0]^T$ 。
2. 设定空间平移位移向量为 $\mathbf{t} = [2.0, 1.5, 2.5]^T$ 。
3. 编写 Python 代码，利用 NumPy 的数组加法计算物体平移后的目标点 B 坐标 $\mathbf{p}_B$ ，并在控制台中格式化输出。
4. （选做）调用提供的平移动画接口，传入平移向量 $\mathbf{t}$ ，观察小飞机模型在保持初始姿态不变的前提下平移至目标位置的动态过程。

1.1.5 实验步骤

1. 使用 `np.array` 分别定义初始点坐标向量 $\mathbf{p}_A$ 与平移向量 $\mathbf{t_vec}$ 。
2. 调用向量加法操作符 `+` 计算目标位置 $\mathbf{p}_B$ 。
3. 打印输出初始点、平移位移量及计算得到的目标点具体数值。
4. （选做）引入 `animator_vec_rotation.py` 中的 `play_step_animation` 函数，设置 `translation_vec=t_vec` 且 `rotation_matrices=None` 播放纯平移动画。

1.1.6 示例代码

```
import numpy as np

#定义初始位置点 A 与平移向量 t(三维空间向量)
p_A = np.array([1.0, 0.5, 0.0], dtype=np.float64)
t_vec = np.array([2.0, 1.5, 2.5], dtype=np.float64)

#执行向量代数加法运算求解目标点 B
p_B = p_A + t_vec
```

#打印输出计算结果

```
print(f'初始点 A 位置坐标: {p_A}')
```

```
print(f'空间平移位移向量 t: {t_vec}')
```

```
print(f'平移变换后目标点 B 坐标: {p_B}')
```

#[选做] 调用动画模块可视化纯平移动态过程

try:

#导入小飞机模块，传入平移矩阵，旋转矩阵置空

```
from animator_3d import play_3d_step_animation
```

```
play_3d_step_animation(translation_vec_inc=t_vec, translation_vec_start=p_A)
```

except ImportError:

```
pass
```

1.1.7. 实验结果

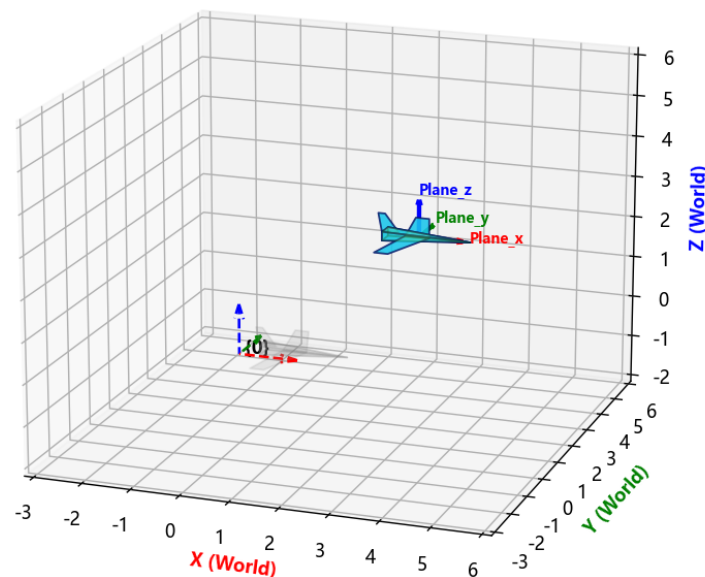
控制台输出：

初始点 A 位置坐标: [1. 0.5 0.]

空间平移位移向量 t: [2. 1.5 2.5]

平移变换后目标点 B 坐标: [3. 2. 2.5]。

图形呈现（选做）：小飞机模型保持初始朝向不变，沿三维直线平滑移动至目标点，空间原点留下半透明参考残影。



1.2 实验二：绕固定基坐标系连续旋转（外旋与矩阵左乘）

1.2.1 实验目的

1. 掌握绕空间绝对参考系（World Frame）各轴旋转的标准旋转矩阵形式。
2. 理解欧拉角（Euler Angles）概念及外旋矩阵左乘原则。

1.2.2 实验环境

1. 软件环境：Python 3.8+
2. 核心依赖包：numpy、matplotlib
3. 工程辅助模块：animator_3d.py

1.2.3 实验原理

旋转矩阵的物理意义与基准点 $[1, 1, 1]^T$

旋转矩阵 $\mathbf{R} \in SO(3)$ 是描述刚体系统“姿态”的数学工具。

当物体位于世界参考系原点且未发生平移时，其几何中心位置为 $\mathbf{p}_{origin} = [0, 0, 0]^T$ 。根据线性代数矩阵乘法定义：

$$\mathbf{p}'_{origin} = \mathbf{R} \cdot \mathbf{p}_{origin} = \mathbf{R} \cdot \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}$$

由于任何旋转矩阵乘以零向量结果恒为零向量，直接对原点坐标进行矩阵乘算无法反映物体的姿态旋转变化。

因此，为了在数值代数上显式度量刚体的空间姿态改变，实验中引入非原点的空间探测基准点 $\mathbf{p}_{test} = [1.0, 1.0, 1.0]^T$ 。通过计算该特征点在旋转变换后的新坐标 $\mathbf{p}' = \mathbf{R} \cdot \mathbf{p}_{test}$ ，即可直观量化物体各维度基底在空间中的偏转程度。

绕固定基坐标轴的标准旋转矩阵

在三维笛卡尔坐标系中，绕固定世界参考坐标系的 X, Y, Z 轴旋转角度 θ 的标准旋转矩阵定义如下：

$$\mathbf{R}_x(\theta) = \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos \theta & -\sin \theta \\ 0 & \sin \theta & \cos \theta \end{bmatrix}, \mathbf{R}_y(\theta) = \begin{bmatrix} \cos \theta & 0 & \sin \theta \\ 0 & 1 & 0 \\ -\sin \theta & 0 & \cos \theta \end{bmatrix}, \mathbf{R}_z(\theta) = \begin{bmatrix} \cos \theta & -\sin \theta & 0 \\ \sin \theta & \cos \theta & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

外旋连续相乘法则（矩阵左乘原则）

外旋是指刚体的每一次旋转始终以静止不动的外部世界基坐标轴（World Fixed Axes）为参考轴。

若刚体依次执行以下动作：

1. 先绕固定世界 X 轴旋转 α (对应矩阵 $\mathbf{R}_x(\alpha)$)；
2. 再绕固定世界 Y 轴旋转 β (对应矩阵 $\mathbf{R}_y(\beta)$)；
3. 最后绕固定世界 Z 轴旋转 γ (对应矩阵 $\mathbf{R}_z(\gamma)$)。

根据多坐标系投影推导，后发生的旋转作用在全局基准上，因此复合旋转矩阵必须采用从左侧依次连乘（左乘）的形式：

$$\mathbf{R}_{ext} = \mathbf{R}_z(\gamma) \cdot \mathbf{R}_y(\beta) \cdot \mathbf{R}_x(\alpha)$$

对于基准探测点 $\mathbf{p}_{test}$ ，旋转后的空间坐标为：

$$\mathbf{p}' = \mathbf{R}_{ext} \cdot \mathbf{p}_{test} = \mathbf{R}_z(\gamma) \cdot \mathbf{R}_y(\beta) \cdot \mathbf{R}_x(\alpha) \cdot \mathbf{p}_{test}$$

NumPy 核心函数与矩阵操作机制：

本实验涉及以下 NumPy 核心语法与数学函数：

1. 角度转弧度函数 `np.deg2rad(deg)`：

- 将以角度制 (Degree, 如 45°) 表示的数值转换为三角函数所需的弧度制 (Radian,

即 $\text{rad} = \text{deg} \times \frac{\pi}{180}$)。

- **形参**：deg, 支持单个整型/浮点型数值或 NumPy 数组。

注意事项: Python 及 NumPy 中的三角函数 `np.cos()`、`np.sin()` 严格要求输入参数为弧度。直接传入角度值会导致计算严重失真。

2. 三角函数 `np.cos(rad)` 与 `np.sin(rad)`：

- **作用**：计算指定弧度值的余弦与正弦。

3. 二维旋转矩阵构建 `np.array([[...], [...], [...]])`：

- 通过向 `np.array` 传入一个包含 3 个子列表的嵌套列表构建 3×3 的二维矩阵，并指定 `dtype=np.float64`。

4. 矩阵乘法运算符 `@`：

在 NumPy 中，`@` 操作符（或 `np.matmul`）专用于执行标准线性代数矩阵乘法。

`R_z @ R_y @ R_x` 会按照矩阵乘法规则计算出复合矩阵；

`R_ext @ p_test` 执行 3×3 矩阵与 3×1 向量的矩阵积，输出 shape 为 (3,) 的变换后坐标向量。**(注：避免使用 `*` 运算符，`*` 在 NumPy 中表示逐元素相乘，无法完成空间坐标变换)。**

5. 数值修约函数 `np.round(arr, decimals=4)`：

- 对浮点矩阵进行四舍五入并保留指定小数位数（如 4 位），便于控制台观察。

1.2.4 实验要求

1. 设定绕固定世界基坐标轴的旋转序列：依次绕世界 X 轴旋转 45.0° 、世界 Y 轴旋转 30.0° 、世界 Z 轴旋转 60.0° 。
2. 编写函数分别构建 $\mathbf{R}_x(45^\circ)$ 、 $\mathbf{R}_y(30^\circ)$ 、 $\mathbf{R}_z(60^\circ)$ 三个 3×3 基础旋转矩阵。

3. 按照外旋左乘法则，计算综合复合旋转矩阵 $\mathbf{R}_{ext}$ 并打印输出。
4. 输入空间探测基准点 $\mathbf{p}_{test} = [1.0, 1.0, 1.0]^T$ ，通过矩阵向量积求解变换后的新坐标 $\mathbf{p}'$ 。
5. （选做）调用 animator_vec_rotation.py 中的 play_step_animation 函数，设置 rotation_matrices_end=[Rx, Ry, Rz] 及 frame_mode='world'，动态观察小飞机绕世界固定轴依次转动的过程。

1.2.5 实验步骤

1. 导入 numpy 模块。
2. 编写单轴旋转矩阵生成函数 rot_x(deg)、rot_y(deg)、rot_z(deg)，内部使用 np.deg2rad 完成弧度转换。
3. 分别实例化 Rx、Ry、Rz。
4. 按照左乘规则执行矩阵相乘：R_ext = Rz @ Ry @ Rx。
5. 定义探测基准点 p_test = np.array([1.0, 1.0, 1.0], dtype=np.float64)，计算 p_prime = R_ext @ p_test 并格式化输出。
6. （选做）引入动画模块，传入三个旋转矩阵列表，指定 frame_mode='world' 播放外旋分步动画。

1.2.6 示例代码

```
import numpy as np

#定义单轴旋转矩阵构造函数

def rot_x(deg):
    """构建绕 X 轴旋转矩阵 (输入参数 deg 为角度制)"""
    r = np.deg2rad(deg) # 角度转弧度
    return np.array([
        [1.0, 0.0, 0.0],
        [0.0, np.cos(r), -np.sin(r)],
        [0.0, np.sin(r), np.cos(r)]
    ], dtype=np.float64)

def rot_y(deg):
    """构建绕 Y 轴旋转矩阵 (输入参数 deg 为角度制)"""
    r = np.deg2rad(deg)
```

```
return np.array([
    [ np.cos(r), 0.0, np.sin(r)],
    [ 0.0,      1.0, 0.0      ],
    [-np.sin(r), 0.0, np.cos(r)]
], dtype=np.float64)
```

```
def rot_z(deg):
```

```
    """构建绕 Z 轴旋转矩阵 (输入参数 deg 为角度制)"""
```

```
    r = np.deg2rad(deg)
    return np.array([
        [np.cos(r), -np.sin(r), 0.0],
        [np.sin(r),  np.cos(r), 0.0],
        [0.0,      0.0,      1.0]
    ], dtype=np.float64)
```

```
#构造基础旋转矩阵
```

```
Rx = rot_x(45.0) # 绕世界 X 轴旋转 45 度
```

```
Ry = rot_y(30.0) # 绕世界 Y 轴旋转 30 度
```

```
Rz = rot_z(60.0) # 绕世界 Z 轴旋转 60 度
```

```
#外旋连乘计算：绕固定世界基坐标系依次旋转 X -> Y -> Z (采用矩阵左乘)
```

```
R_ext = Rz @ Ry @ Rx
```

```
#空间探测基准点变换计算
```

```
# 引入非零基准点以反映姿态变化 (若为 [0,0,0]^T 则旋转前后恒为 [0,0,0]^T)
```

```
p_test = np.array([1.0, 1.0, 1.0], dtype=np.float64)
```

```
p_prime = R_ext @ p_test
```

```
#控制台输出计算结果
```

```
print("单轴旋转矩阵 Rx (45°):\n", np.round(Rx, 4))
```

```
print("\n 单轴旋转矩阵 Ry (30°):\n", np.round(Ry, 4))
```

```
print("\n 单轴旋转矩阵 Rz (60°):\n", np.round(Rz, 4))
```

```
print("\n 复合外旋总矩阵 R_ext = Rz @ Ry @ Rx (3x3):\n", np.round(R_ext, 4))
```

```
print("经复合外旋变换后的空间点 p_prime 坐标: ", np.round(p_prime, 4))

#[选做] 调用分步动画模块可视化外旋过程

try:
    from animator_3d import play_3d_step_animation
    play_3d_step_animation(
        rotation_matrices_inc=[Rx, Ry, Rz],    # 传入 3 个基础旋转矩阵
        frame_mode='world',                    # 指定为世界基坐标系外旋模式 (矩阵左乘)
        order='rotate_first'
    )
except ImportError:
    pass
```

1.2.7 实验结果

控制台输出：

单轴旋转矩阵 Rx (45°):

```
[[ 1.    0.    0.   ]
 [ 0.    0.7071 -0.7071]
 [ 0.    0.7071  0.7071]]
```

单轴旋转矩阵 Ry (30°):

```
[[ 0.866  0.    0.5 ]
 [ 0.    1.    0.   ]
 [-0.5   0.    0.866]]
```

单轴旋转矩阵 Rz (60°):

```
[[ 0.5  -0.866  0.   ]
 [ 0.866  0.5   0.   ]
 [ 0.    0.    1.   ]]
```

复合外旋总矩阵 R_ext = Rz @ Ry @ Rx (3x3):

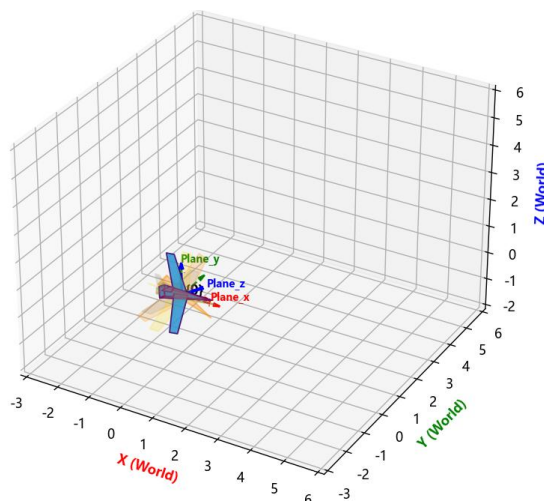
```
[[ 0.433  -0.4356  0.7891]
 [ 0.75    0.6597 -0.0474]
```


[-0.5 0.6124 0.6124]]

空间探测基准点 p_test 初始坐标: [1. 1. 1.]

经复合外旋变换后的空间点 p_prime 坐标: [0.7866 1.3624 0.7247]

选做可视化窗口结果:



1.3 实验三：绕运动局部坐标系连续旋转（内旋与矩阵右乘）

1.3.1 实验目的

1. 理解内旋的物理运动机制与随动参考系演变过程。
2. 掌握物体绕自身局部坐标轴旋转时旋转矩阵连乘的“右乘法则”及其数学推导逻辑。
3. 对比内旋（右乘）与外旋（左乘）在相同旋转角输入下的代数差异与终态姿态差异，了解三维空间旋转群 $SO(3)$ 非对易性的认知。
4. 掌握利用 NumPy 执行矩阵右乘链式运算

1.3.2 实验环境

1. **软件环境：**Python 3.8+
2. **核心依赖包：**numpy、matplotlib
3. **工程辅助模块：**animator_3d.py

1.3.3 实验原理

内旋与随动局部坐标系的物理机制

与实验三中参考轴始终固定不动的“外旋”不同，内旋是指刚体的每一次旋转始终以当前物体自身所固连的、刚刚完成上一级旋转后形成的最新局部坐标轴为基准进行转动。

在航空航天与机器人学中，飞行器的姿态调整（Roll 滚转、Pitch 俯仰、Yaw 偏航）本质上均属于内旋过程。例如：飞机在做完一次机头偏转后，接下来的抬头动作必然是沿着偏转后指向机身侧向的新机翼轴线向上抬头，而非沿着地面固定的水平轴抬头。

内旋连续相乘

设飞机 P 的质心初始时刻与基坐标系 {0} 完全重合：

第 1 步： P 绕自身初始机体轴 X 旋转 α 角，形成第 1 代中间坐标系 {1}，其相对于基坐标系 {0} 的姿态为：

$$\mathbf{R}_1 = \mathbf{R}_x(\alpha)$$

第 2 步： P 绕当前最新的自身机体轴 Y' 旋转 β 角，形成第 2 代中间坐标系 {2}。由于这次旋转发生在局部坐标系内部，根据坐标变换的基底投影定理，新旋转算子必须作用于前一状态的右侧：

$$\mathbf{R}_2 = \mathbf{R}_1 \cdot \mathbf{R}_y(\beta) = \mathbf{R}_x(\alpha) \cdot \mathbf{R}_y(\beta)$$

第 3 步： P 绕最新形成的机体轴 Z'' 旋转 γ 角，形成最终姿态坐标系 {3}。同理，变换矩阵继续从右侧连乘：

$$\mathbf{R}_{int} = \mathbf{R}_2 \cdot \mathbf{R}_z(\gamma) = \mathbf{R}_x(\alpha) \cdot \mathbf{R}_y(\beta) \cdot \mathbf{R}_z(\gamma)$$

外旋与内旋的核心代数对比：

绕外部固定基坐标系连续旋转 $\Rightarrow$ 矩阵从左侧依次连乘（左乘）：

$$\mathbf{R}_{ext} = \mathbf{R}_{last} \cdots \mathbf{R}_2 \cdot \mathbf{R}_1$$

绕自身运动局部坐标系连续旋转 $\Rightarrow$ 矩阵从右侧依次连乘（右乘）：

$$\mathbf{R}_{int} = \mathbf{R}_1 \cdot \mathbf{R}_2 \cdots \mathbf{R}_{last}$$

旋转的非对易性（ $SO(3)$ 代数特征）

在三维空间中，除了共轴旋转等特例外，三维旋转算子一般不满足乘法交换律：

$$\mathbf{R}_A \cdot \mathbf{R}_B \neq \mathbf{R}_B \cdot \mathbf{R}_A$$

因此，即使给定的各轴旋转角度与顺序完全相同（如均为 $X(45^\circ) \rightarrow Y(30^\circ) \rightarrow Z(60^\circ)$ ），外旋总矩阵与内旋总矩阵在数值上是截然不同的：

$$\mathbf{R}_{ext} = \mathbf{R}_z(60^\circ)\mathbf{R}_y(30^\circ)\mathbf{R}_x(45^\circ) \neq \mathbf{R}_{int} = \mathbf{R}_x(45^\circ)\mathbf{R}_y(30^\circ)\mathbf{R}_z(60^\circ)$$

探测基准点 $[1, 1, 1]^T$ 的姿态量化

同实验三所述，由于刚体原点 $[0, 0, 0]^T$ 乘算旋转矩阵结果恒为 $[0, 0, 0]^T$ ，无法反映内旋引起的轴向偏转。故继续引入空间非零探测基准点 $\mathbf{p}_{test} = [1.0, 1.0, 1.0]^T$ ，通过计算：

$$\mathbf{p}'_{int} = \mathbf{R}_{int} \cdot \mathbf{p}_{test}$$

由于初始位置没有位移偏量，所以点位 P 将不会改变。在实验 5 中，我们可以更清晰的看到当有位移偏量的旋转时

1.3.4 实验要求

1. 设定与实验三相同的单轴转角输入：绕自身 X 轴旋转 45.0° 、绕自身 Y' 轴旋转 30.0° 、绕自身 Z'' 轴旋转 60.0° 。
2. 编写函数构建基础旋转矩阵 $\mathbf{R}_x(45^\circ)$ 、 $\mathbf{R}_y(30^\circ)$ 、 $\mathbf{R}_z(60^\circ)$ 。

- 按照内旋右乘法则，通过矩阵乘法运算符 @ 计算综合复合内旋矩阵 $\mathbf{R}_{int} = \mathbf{R}_x \cdot \mathbf{R}_y \cdot \mathbf{R}_z$ 。
- (选做) 调用 animator_vec_rotation.py 中的 play_step_animation 函数，设置 rotation_matrices_end=[Rx, Ry, Rz] 及 frame_mode='body'，动态观察小飞机沿随动机身轴依次旋转的物理过程，对比 frame_mode='body' 与 frame_mode='world' 引起的物理航向差异。

1.3.5 实验步骤

- 导入 numpy 模块。
- 构造基础单轴旋转函数 rot_x(deg)、rot_y(deg)、rot_z(deg)。
- 实例化三个基础矩阵 Rx、Ry、Rz。
- 按照右乘顺序执行相乘：R_int = Rx @ Ry @ Rz；同时计算外旋矩阵 R_ext = Rz @ Ry @ Rx。
- 定义基准点 p_test，计算并打印 p_prime_int = R_int @ p_test。
- (选做) 引入动画模块，传入内旋矩阵列表并指定参数 frame_mode='body' 启动内旋分步动画，同时可以传入外旋矩阵并对比两动画的区别。

1.3.6 示例代码

```
import numpy as np

#定义基础单轴旋转矩阵构造函数
def rot_x(deg):
    """构建绕 X 轴旋转矩阵 (输入参数 deg 为角度制)"""
    r = np.deg2rad(deg)
    return np.array([
        [1.0, 0.0, 0.0],
        [0.0, np.cos(r), -np.sin(r)],
        [0.0, np.sin(r), np.cos(r)]
    ], dtype=np.float64)

def rot_y(deg):
    """构建绕 Y 轴旋转矩阵 (输入参数 deg 为角度制)"""
    r = np.deg2rad(deg)
    return np.array([
        [ np.cos(r), 0.0, np.sin(r)],
```

```
        [ 0.0,      1.0, 0.0      ],  
        [-np.sin(r), 0.0, np.cos(r)]  
    ], dtype=np.float64)
```

```
def rot_z(deg):
```

```
    """构建绕 Z 轴旋转矩阵 (输入参数 deg 为角度制)"""
```

```
    r = np.deg2rad(deg)  
    return np.array([  
        [np.cos(r), -np.sin(r), 0.0],  
        [np.sin(r),  np.cos(r), 0.0],  
        [0.0,      0.0,      1.0]  
    ], dtype=np.float64)
```

```
#构造基础旋转矩阵
```

```
Rx = rot_x(45.0) # 绕机体自身 X 轴旋转 45 度 (Roll 滚转)  
Ry = rot_y(30.0) # 绕机体自身 Y' 轴旋转 30 度 (Pitch 俯仰)  
Rz = rot_z(60.0) # 绕机体自身 Z'' 轴旋转 60 度 (Yaw 偏航)
```

```
#复合旋转矩阵计算：内旋右乘 vs 外旋左乘
```

```
#内旋（绕自身运动局部坐标轴）：依次右乘 Rx @ Ry @ Rz  
R_int = Rx @ Ry @ Rz
```

```
#外旋（绕固定世界基坐标轴）：依次左乘 Rz @ Ry @ Rx  
R_ext = Rz @ Ry @ Rx
```

```
#空间探测基准点变换计算
```

```
p_test = np.array([1.0, 1.0, 1.0], dtype=np.float64)  
p_prime_int = R_int @ p_test  
p_prime_ext = R_ext @ p_test
```

```
#控制台输出与代数非对易性校验
```

```
print("复合内旋总矩阵 R_int = Rx @ Ry @ Rz (3x3):\n", np.round(R_int, 4))  
print("\n 复合外旋总矩阵 R_ext = Rz @ Ry @ Rx (3x3):\n", np.round(R_ext, 4))
```

#[选做] 调用分步动画模块可视化内旋过程

try:

```
from animator_3d import play_3d_step_animation
```

#列表形式传入旋转矩阵动画按照 Rx->Ry->Rz 顺序旋转，但一个是绕局部坐标系一个绕世界坐标系

```
play_3d_step_animation(
```

```
    rotation_matrices_inc=[Rx, Ry, Rz],    #传入 3 个基础旋转矩阵
```

```
    #rotation_matrices_inc=R_int           #(可选打开注释)传入 1 个复合旋转矩阵
```

```
    frame_mode='body',                    #指定为机身坐标系内旋模式 (矩阵右乘)
```

```
    order='rotate_first'
```

```
)
```

```
play_3d_step_animation(
```

```
    rotation_matrices_inc=[Rx, Ry, Rz],    #传入 3 个基础旋转矩阵
```

```
    #rotation_matrices_inc=R_ext           #(可选打开注释)传入 1 个复合旋转矩阵
```

```
    frame_mode='world',                    #指定为世界坐标系外旋模式 (矩阵左乘)
```

```
    order='rotate_first'
```

```
)
```

```
except ImportError:
```

```
    pass
```

1.3.7 实验结果

控制台输出：

复合内旋总矩阵 $R_{int} = R_x @ R_y @ R_z$ (3x3):

```
[[ 0.433  -0.75    0.5    ]
```

```
 [ 0.7891  0.0474 -0.6124]
```

```
 [ 0.4356  0.6597  0.6124]]
```

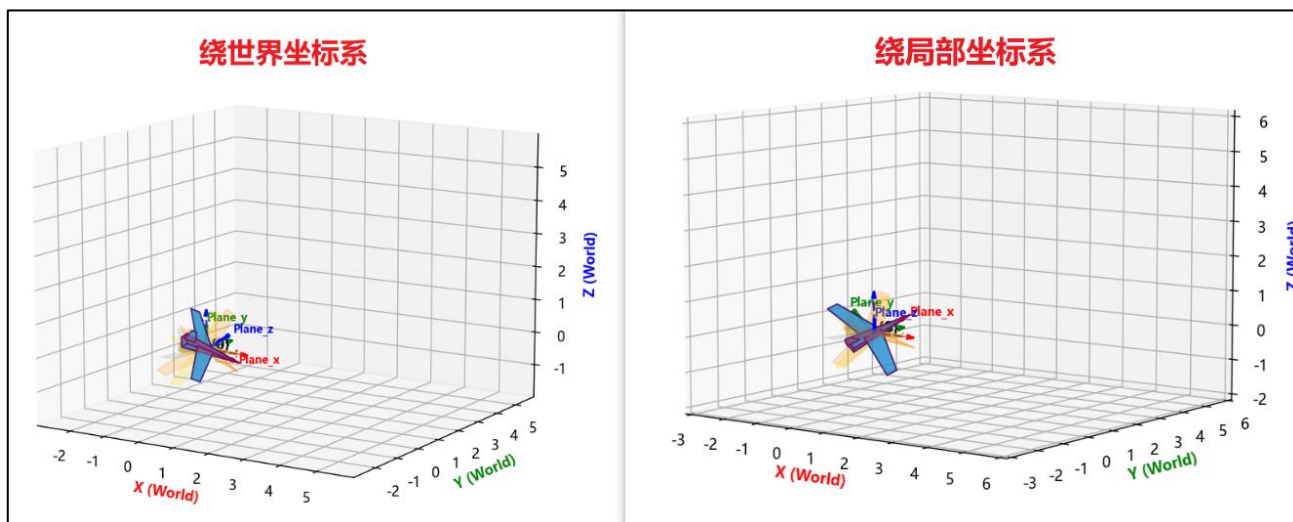
复合外旋总矩阵 $R_{ext} = R_z @ R_y @ R_x$ (3x3):

```
[[ 0.433  -0.4356  0.7891]
```

```
 [ 0.75    0.6597 -0.0474]
```

```
 [-0.5    0.6124  0.6124]]
```

图形呈现（选做）：



1.4 实验四：复合运动旋转顺序先后对比

1.4.1. 实验目的

1. 从矩阵代数与空间几何双重视角，深入理解物体在三维空间运动中平移算子与旋转算子的非对易耦合性。
2. 掌握空间几何中标准的位姿构建方式——“先旋转（在原点自转调姿），后平移（直线移动定位）”的数学本质与设计初衷。
3. 深刻理解并量化“先平移后绕世界轴旋转”导致空间点位发生偏移的几何机理。

1.4.2. 实验环境

1. 软件环境：Python 3.8+
2. 核心依赖包：numpy、matplotlib
3. 工程辅助模块：animator_3d.py

1.4.3. 实验原理

先旋转，后平移（标准位姿构建方式）

在描述物体在世界坐标系 $\{0\}$ 下的位置 $\mathbf{t}$ 与姿态 $\mathbf{R}$ 时，标准的运算次序定义为：物体首先在原点自转调整朝向，随后整体平移至指定目标坐标。

数学推导过程如下：

第 1 阶段（自转）：物体在世界原点处执行纯旋转 $\mathbf{R}$ 。由于物体初始中心处于原点 $[0,0,0]^T$ ，且满足：

$$\mathbf{R} \cdot \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}$$

旋转过程完全独立，不产生任何位置位移，仅改变物体自身的坐标轴朝向；

第 2 阶段（平移）：物体沿目标位移矢量 $\mathbf{t}$ 执行纯平移。

最终物体几何中心的绝对空间坐标为：

$$\mathbf{p}_{final}^{(1)} = \mathbf{R} \cdot \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix} + \mathbf{t} = \mathbf{t}$$

先平移，后绕世界轴旋转（颠倒次序下的位移偏移现象）

若将操作次序颠倒：物体首先沿 $\mathbf{t}$ 平移，随后在世界基坐标系下执行旋转 $\mathbf{R}$ 。数学推导过程如下：

第 1 阶段（平移）：物体几何中心平移至坐标 $\mathbf{p} = \mathbf{t}$ ；

第 2 阶段（外旋）：物体绕外部世界基坐标系原点旋转 $\mathbf{R}$ 。由于此时物体与世界原点之间已经存在平移距离 $\mathbf{t}$ ，世界坐标系下的旋转矩阵会将该平移矢量一同旋转变换为 $\mathbf{Rt}$ 。

最终物体几何中心的空间坐标变为：

$$\mathbf{p}_{final}^{(2)} = \mathbf{R} \cdot \left(\begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix} + \mathbf{t} \right) = \mathbf{Rt}$$

误差代数差值分析

两种时序下物体中心位置的代数差值为：

$$\Delta \mathbf{p} = \mathbf{p}_{final}^{(1)} - \mathbf{p}_{final}^{(2)} = \mathbf{t} - \mathbf{Rt} = (\mathbf{I}_{3 \times 3} - \mathbf{R}) \cdot \mathbf{t}$$

只有当 $\mathbf{t} = \mathbf{0}$ （无平移）或 $\mathbf{R} = \mathbf{I}$ （无旋转）或平移矢量恰好落在旋转轴上时， $\Delta \mathbf{p}$ 才为 $\mathbf{0}$ ；在绝大多数情况下， $(\mathbf{I} - \mathbf{R})\mathbf{t} \neq \mathbf{0}$ 。这证明了平移与旋转在时间先后顺序上是不具备可交换性的。

1.4.4. 实验要求

1. 设定目标平移向量为 $\mathbf{t} = [2.5, 1.5, 2.0]^T$ 。
2. 设定三次旋转角度参数：绕 X 轴旋转 45.0° 、绕 Y 轴旋转 30.0° 、绕 Z 轴旋转 60.0° 。
3. 编写函数分别构建基础旋转矩阵 $\mathbf{R}_x(45^\circ)$ 、 $\mathbf{R}_y(30^\circ)$ 、 $\mathbf{R}_z(60^\circ)$ 。

先旋转后平移下，物体几何中心最终坐标 $\mathbf{p}_{final}^{(1)}$ ；

先平移后绕世界轴旋转下，物体几何中心最终坐标 $\mathbf{p}_{final}^{(2)}$ ；

4. （选做）调用 `animator_vec_rotation.py` 中的 `play_step_animation` 函数，分别指定 `order='rotate_first'` 与 `order='translate_first'` (`frame_mode='world'`)，动态观察两种时序下小飞机实际运动轨迹与最终停留位置。

1.4.5. 实验步骤

1. 导入 `numpy` 模块。

2. 定义平移向量 $t_vec = np.array([2.5, 1.5, 2.0])$ 与 90° 旋转矩阵 R_z 。
3. 按照原理公式计算 $p_final_1 = t_vec$ 与 $p_final_2 = R_z @ t_vec$ 。
4. (选做) 调用动画模块, 先后播放两种时序的动画。

1.4.6. 示例代码

```
import numpy as np

#定义基础单轴旋转矩阵构造函数

def rot_x(deg):
    """构建绕 X 轴旋转矩阵 (输入参数 deg 为角度制)"""
    r = np.deg2rad(deg)
    return np.array([
        [1.0, 0.0, 0.0],
        [0.0, np.cos(r), -np.sin(r)],
        [0.0, np.sin(r), np.cos(r)]
    ], dtype=np.float64)

def rot_y(deg):
    """构建绕 Y 轴旋转矩阵 (输入参数 deg 为角度制)"""
    r = np.deg2rad(deg)
    return np.array([
        [np.cos(r), 0.0, np.sin(r)],
        [0.0, 1.0, 0.0],
        [-np.sin(r), 0.0, np.cos(r)]
    ], dtype=np.float64)

def rot_z(deg):
    """构建绕 Z 轴旋转矩阵 (输入参数 deg 为角度制)"""
    r = np.deg2rad(deg)
    return np.array([
        [np.cos(r), -np.sin(r), 0.0],
```



```
        [np.sin(r), np.cos(r), 0.0],  
        [0.0, 0.0, 1.0]  
    ], dtype=np.float64)
```

#参数定义

#目标平移向量

```
t_vec = np.array([2.5, 1.5, 2.0], dtype=np.float64)
```

#构造 X -> Y -> Z 三次连续旋转的基础矩阵

```
Rx = rot_x(45.0) # 绕 X 轴旋转 45 度
```

```
Ry = rot_y(30.0) # 绕 Y 轴旋转 30 度
```

```
Rz = rot_z(60.0) # 绕 Z 轴旋转 60 度
```

#计算复合外旋总矩阵 (世界基轴左乘: Rz @ Ry @ Rx)

```
R_ext = Rz @ Ry @ Rx
```

#两种时序下的物体几何中心位置代数计算

#先在原点经历三次连续自转，后平移定位

```
p_final_1 = t_vec.copy()
```

#先平移至 t_vec，后绕世界基坐标轴经历三次连续外旋

```
p_final_2 = R_ext @ t_vec
```

#控制台输出计算与对比结果

```
print(f'目标平移向量 t: {t_vec}')
```

```
print("\n 复合外旋总矩阵 R_ext(Rz @ Ry @ Rx):\n", np.round(R_ext, 4))
```

```
print(f'\n (先旋转后平移)最终中心坐标: {np.round(p_final_1, 4)}')
```

```
print(f'(先平移后外旋)最终中心坐标: {np.round(p_final_2, 4)}')
```

#调用动画模块可视化对比

try:

```
from animator_3d import play_3d_step_animation
```

```
play_3d_step_animation(  
    translation_vec_inc=t_vec,  
    rotation_matrices_inc=[Rx, Ry, Rz],  
    order='rotate_first',  
    frame_mode='body'  
)
```

```
play_3d_step_animation(  
    translation_vec_inc=t_vec,  
    rotation_matrices_inc=[Rx, Ry, Rz],  
    order='translate_first',  
    frame_mode='world'  
)
```

except ImportError:

```
pass
```

1.4.7. 实验结果

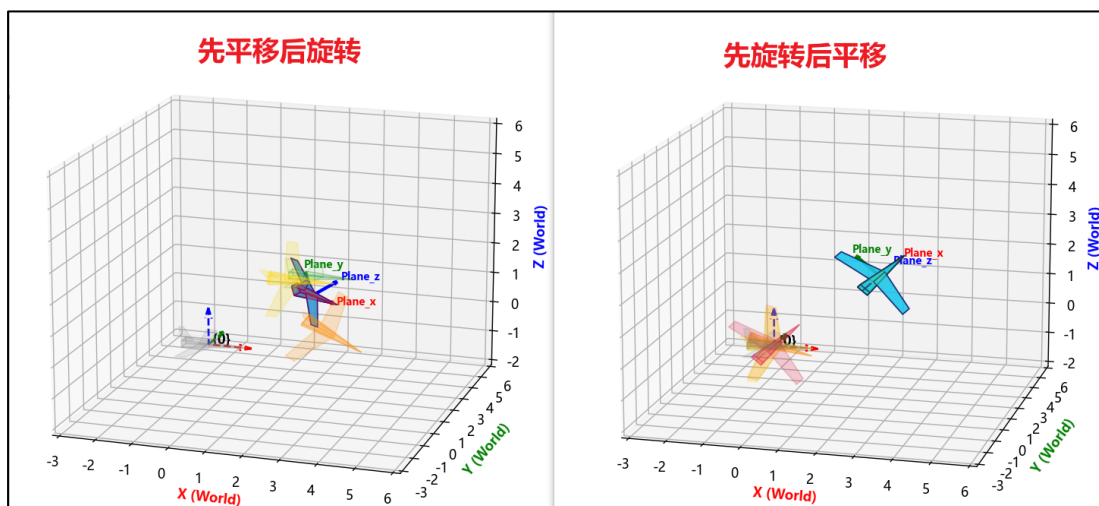
控制台输出：

(先旋转后平移 / 标准模式) 最终中心坐标: [2.5 1.5 2.]

(先平移后外旋 / 颠倒模式) 最终中心坐标: [2.0074 2.7699 0.8933]

图形

图形呈现（选做）：



1.5 实验五：跨坐标系位姿映射与相对运动学表达

1.5.1 实验目的

1. 理解多参考系环境下，空间几何点在不同坐标系之间的相对坐标转换与空间映射原理。
2. 理解参考坐标系 $\{A\}$ 在全局世界坐标系 $\{0\}$ 下的描述。
3. 理解在局部参考坐标系内部完成“先外旋后平移”的复合运动后，将目标物体在局部坐标系中的点位映射为世界绝对坐标的代数推导过程。
4. 运用 NumPy 矩阵乘法 ($@$) 与向量加法完成多坐标系下的层级几何解算。

1.5.2 实验环境

1. 软件环境：Python 3.8+
2. 核心依赖包：numpy、matplotlib
3. 工程辅助模块：animator_3d.py

1.5.3 实验原理

跨参考坐标系的几何映射

在三维空间中，设全局静止基准参考系为世界坐标系 $\{0\}$ ，空间中建立了一个新的局部参考坐标系 $\{A\}$ 。坐标系 $\{A\}$ 相对于世界坐标系 $\{0\}$ 的空间几何关系由以下两项唯一确定：

姿态关系：通过旋转矩阵 ${}^0\mathbf{R}_A \in SO(3)$ 描述坐标系 $\{A\}$ 各正交轴相对于世界坐标系轴向的投影偏转；

位置关系：通过原点偏移向量 ${}^0\mathbf{t}_A \in \mathbb{R}^3$ 描述坐标系 $\{A\}$ 的原点在世界坐标系下的三维空间位

置。

若已知空间某物体上的几何点在局部坐标系 $\{A\}$ 下的局部坐标为 ${}^A\mathbf{p} = [x_A, y_A, z_A]^T$ ，则该点在世界基坐标系 $\{0\}$ 下的绝对空间坐标 ${}^0\mathbf{p}$ 满足以下线性映射方程：

$${}^0\mathbf{p} = {}^0\mathbf{R}_A \cdot {}^A\mathbf{p} + {}^0\mathbf{t}_A$$

该公式表明：将局部点投影到世界坐标系时，首先利用旋转矩阵 ${}^0\mathbf{R}_A$ 将局部轴向分量旋转对齐到世界基轴方向，再加上两坐标系原点之间的平移偏量 ${}^0\mathbf{t}_A$ 。

局部参考系内部的复合运动（先外旋后平移）

当物体在局部参考坐标系 $\{A\}$ 内部经历初始放置与二次运动时，其运算遵循以下两阶段：

初始状态定义：

物体初始处于坐标系 $\{A\}$ 中的点位 ${}^A\mathbf{p}_{start}$ ，初始姿态为 ${}^A\mathbf{R}_{start}$ 。

在坐标系 $\{A\}$ 内部发生运动（先外旋后平移）：

物体相对于坐标系 $\{A\}$ 的基准轴依次经历绕 $X_A \rightarrow Y_A \rightarrow Z_A$ 的三次连续外旋，对应的复合外旋矩阵为：

$${}^A\mathbf{R}_{inc} = \mathbf{R}_z(\gamma) \cdot \mathbf{R}_y(\beta) \cdot \mathbf{R}_x(\alpha)$$

我们采用先外旋后平移的标准时序，外旋矩阵首先作用于初始点位，随后叠加局部平移位移增量 ${}^A\mathbf{t}_{inc}$ 。物体在坐标系 $\{A\}$ 内部的最终空间点位为：

$${}^A\mathbf{p}_{final} = {}^A\mathbf{R}_{inc} \cdot {}^A\mathbf{p}_{start} + {}^A\mathbf{t}_{inc}$$

世界绝对坐标合成推导

将局部计算得到的目标点位 ${}^A\mathbf{p}_{final}$ 代入跨坐标系投影公式，即可求出物体最终在世界基坐标系 $\{0\}$ 下的绝对三维坐标：

$${}^0\mathbf{p}_{final} = {}^0\mathbf{R}_A \cdot {}^A\mathbf{p}_{final} + {}^0\mathbf{t}_A = {}^0\mathbf{R}_A \cdot ({}^A\mathbf{R}_{inc} \cdot {}^A\mathbf{p}_{start} + {}^A\mathbf{t}_{inc}) + {}^0\mathbf{t}_A$$

1.5.4 实验要求

1.坐标系与点位初始化：

参数设定 1（参考坐标系 $\{A\}$ 的构建）：

- 1) 坐标系 $\{A\}$ 相对于世界坐标系 $\{0\}$ 的原点平移为 ${}^0\mathbf{t}_A = [2.0, 1.5, 1.0]^T$ ；
- 2) 姿态关系为绕世界坐标轴依次旋转： $X(0^\circ) \rightarrow Y(30.0^\circ) \rightarrow Z(45.0^\circ)$ ，构造旋转矩阵 ${}^0\mathbf{R}_A = \mathbf{R}_z(45^\circ)\mathbf{R}_y(30^\circ)\mathbf{R}_x(0^\circ)$ 。

参数设定 2（物体在 $\{A\}$ 中的初始位姿）：

- 1) 物体初始在坐标系 $\{A\}$ 下的位置为 ${}^A\mathbf{p}_{start} = [1.0, 0.5, 0.2]^T$ ，初始姿态为绕 Z_A 轴偏转 15.0° (${}^A\mathbf{R}_{start} = \mathbf{R}_z(15^\circ)$)。

参数设定 3（物体在 $\{A\}$ 中的运动增量）：

- 1) 相对坐标系 $\{A\}$ 依次绕 $X_A \rightarrow Y_A \rightarrow Z_A$ 轴执行外旋： $X(30.0^\circ) \rightarrow Y(20.0^\circ) \rightarrow Z(45.0^\circ)$ ，构造旋转增量矩阵 ${}^A\mathbf{R}_{inc}$ ；
- 2) 相对坐标系 $\{A\}$ 的平移增量为 ${}^A\mathbf{t}_{inc} = [1.2, 0.8, 0.5]^T$ ；
运动时序遵循“先外旋，后平移”。

2. 计算任务:

计算物体运动完成后在局部参考坐标系 $\{A\}$ 下的最终坐标 ${}^A\mathbf{p}_{final}$;

计算物体最终在全局世界坐标系 $\{0\}$ 下的绝对坐标 ${}^0\mathbf{p}_{final}$ 。

3. **(选做任务)**: 调用 `animator_vec_rotation.py` 中的 `play_step_animation` 函数, 传入 `reference_frames` 参数, 动态可视化坐标系 $\{A\}$ 与小飞机的跨坐标系运动演变。

1.5.5 实验步骤

1. 导入 `numpy` 模块并定义单轴旋转函数 `rot_x(deg)`、`rot_y(deg)`、`rot_z(deg)`。
2. 构造坐标系 $\{A\}$ 的旋转矩阵 R_{0_A} 与平移向量 t_{0_A} 。
3. 定义物体在 $\{A\}$ 中的初始位置 p_{start_A} 与初始姿态 R_{start_A} 。
4. 构造相对外旋矩阵 $R_{inc_A} = R_{inc_z} @ R_{inc_y} @ R_{inc_x}$, 定义平移增量 t_{inc_A} 。
5. 按照先外旋后平移计算: $p_{final_A} = R_{inc_A} @ p_{start_A} + t_{inc_A}$ 。
6. 将局部坐标映射为世界绝对坐标: $p_{final_world} = R_{0_A} @ p_{final_A} + t_{0_A}$ 。
7. (选做) 配置 `frame_A` 字典并传入动画函数, 播放多坐标系动态演示。

1.5.6 示例代码

```
import numpy as np

#定义单轴旋转矩阵构造函数
def rot_x(deg):
    """构建绕 X 轴旋转矩阵 (输入参数 deg 为角度制)"""
    r = np.deg2rad(deg)
    return np.array([
        [1.0, 0.0, 0.0],
        [0.0, np.cos(r), -np.sin(r)],
        [0.0, np.sin(r), np.cos(r)]
    ], dtype=np.float64)

def rot_y(deg):
    """构建绕 Y 轴旋转矩阵 (输入参数 deg 为角度制)"""
    r = np.deg2rad(deg)
```

```
return np.array([
    [np.cos(r), 0.0, np.sin(r)],
    [0.0, 1.0, 0.0],
    [-np.sin(r), 0.0, np.cos(r)]
], dtype=np.float64)
```

```
def rot_z(deg):
    """构建绕 Z 轴旋转矩阵 (输入参数 deg 为角度制)"""
    r = np.deg2rad(deg)
    return np.array([
        [np.cos(r), -np.sin(r), 0.0],
        [np.sin(r), np.cos(r), 0.0],
        [0.0, 0.0, 1.0]
    ], dtype=np.float64)
```

#构建参考坐标系{A}相对于世界坐标系{0}的位姿

#坐标系{A}原点在世界坐标系中的位置

```
t_0_A = np.array([2.0, 1.5, 1.0], dtype=np.float64)
```

#坐标系{A}姿态：绕世界轴外旋 $X(0^\circ) \rightarrow Y(30^\circ) \rightarrow Z(45^\circ)$

```
R_0_A = rot_z(45.0) @ rot_y(30.0) @ rot_x(0.0)
```

#定义物体在坐标系{A}中的初始状态与运动增量

#初始点位与初始姿态(相对坐标系{A})

```
p_start_A = np.array([1.0, 0.5, 0.2], dtype=np.float64)
```

```
R_start_A = rot_z(15.0)
```

#运动增量：相对坐标系{A}轴外旋 $X(30^\circ) \rightarrow Y(20^\circ) \rightarrow Z(45^\circ)$

```
R_inc_x = rot_x(30.0)
```

```
R_inc_y = rot_y(20.0)
```

```
R_inc_z = rot_z(45.0)
```

```
R_inc_A = R_inc_z @ R_inc_y @ R_inc_x # 外旋左乘
```

#平移位移增量(相对坐标系 {A})

```
t_inc_A = np.array([1.2, 0.8, 0.5], dtype=np.float64)
```

#坐标解算：局部坐标最终点位->全局世界坐标映射

#计算在坐标系{A}内部经历“先外旋后平移”后的最终局部坐标

$p_{final_A} = R_{inc_A} @ p_{start_A} + t_{inc_A}$

#跨坐标系线性映射：求解在世界基坐标系{0}下的绝对三维坐标

$p_{final_world} = R_{0_A} @ p_{final_A} + t_{0_A}$

#控制台输出计算结果

print("坐标系 {A} 相对世界基坐标系的旋转矩阵 R_0_A (3x3):\n", np.round(R_0_A, 4))

print("坐标系 {A} 相对世界基坐标系的原点平移 t_0_A:", t_0_A)

print("\n 物体在坐标系 {A} 内部的复合旋转矩阵 R_inc_A:\n", np.round(R_inc_A, 4))

print(f'物体在坐标系 {{A}} 内部的初始点位 p_start_A: {p_start_A}')

print(f'物体在坐标系 {{A}} 内部的平移与旋转后的点位 p_final_A:{np.round(p_final_A, 4)}')

print(f'\n 物体最终在全局世界坐标系 {{0}} 下的绝对坐标:{np.round(p_final_world, 4)}')

#[选做]调用动画模块可视化多坐标系及相对运动过程

try:

from animator_3d import play_3d_step_animation

#定义参考坐标系 A

frame_A = {

 't': t_0_A,

 'R': R_0_A,

 'label': '{A}'#参考坐标系名称

}

play_3d_step_animation(

 reference_frames=[frame_A], #建立参考坐标系{A}

 translation_vec_start=p_start_A, #基于坐标系{A}的初始位置

 rotation_matrices_start=R_start_A, #基于坐标系{A}的初始姿态

 translation_vec_inc=t_inc_A, #基于坐标系{A}的平移增量

 rotation_matrices_inc=[R_inc_x, R_inc_y, R_inc_z], #基于坐标系{A}的旋转增量

 order='rotate_first', #先旋转，后平移

```

        frame_mode='world', #相对坐标系{A}的基准轴外旋
        xlim=(-1, 6), ylim=(-1, 6), zlim=(-1, 6),
        frames_per_step=30,
        fps=30
    )
except ImportError:
    pass

```

1.5.7 实验结果

控制台输出：

坐标系 {A} 相对世界基坐标系的旋转矩阵 R_{0_A} (3x3):

```

[[ 0.6124 -0.7071  0.3536]
 [ 0.6124  0.7071  0.3536]
 [-0.5      0.      0.866 ]]

```

坐标系 {A} 相对世界基坐标系的原点平移 t_{0_A} : [2. 1.5 1.]

物体在坐标系 {A} 内部的复合旋转矩阵 R_{inc_A} :

```

[[ 0.6645 -0.4915  0.563 ]
 [ 0.6645  0.7333 -0.1441]
 [-0.342   0.4698  0.8138]]

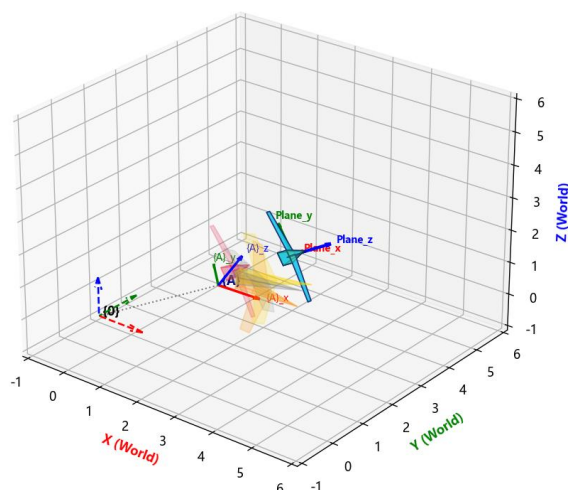
```

物体在坐标系 {A} 内部的初始点位 p_{start_A} : [1. 0.5 0.2]

物体在坐标系 {A} 内部的平移与旋转后的点位 p_{final_A} : [1.7313 1.8023 0.5557]

物体最终在全局世界坐标系 {0} 下的绝对坐标: [1.9823 4.0311 0.6155]

图形呈现（选做）：



1.6 实验六：4 × 4 齐次变换矩阵的构建与统一映射

1.6.1 实验目的

1. 理解三维空间运动学中引入齐次坐标（Homogeneous Coordinates）与 4 × 4 齐次变换矩阵（Homogeneous Transformation Matrix）的核心数学动机。
2. 掌握将 3 × 3 旋转矩阵 $\mathbf{R}$ 与 3 × 1 平移向量 $\mathbf{t}$ 拼接组装为 4 × 4 齐次变换矩阵 $\mathbf{T}$ 的方法。
3. 理解空间几何变换如何通过齐次坐标统一为单一的线性矩阵乘法 ($\tilde{\mathbf{p}}' = \mathbf{T} \cdot \tilde{\mathbf{p}}$)，消除传统变换中“既有乘法又有加法”的非齐次形式。
4. 熟练运用 NumPy 数组切片与赋值语法构建 4 × 4 矩阵，完成齐次坐标扩展与降维提取。
5. （选做）调用齐次矩阵直变动画模块 `play_direct_matrix_animation`，动态观察物体在目标齐次变换矩阵驱动下一步到位的平滑连续飞行动画。

1.6.2 实验环境

1. 软件环境：Python 3.8+
2. 核心依赖包：numpy、matplotlib
3. 工程辅助模块：animator_3d.py

1.6.3 实验原理

从仿射变换到齐次变换的统一

在前面的实验中，将物体局部坐标系 $\{B\}$ 下的点 ${}^B\mathbf{p}$ 转换到世界坐标系 $\{0\}$ 下的坐标 ${}^0\mathbf{p}$ ，采用的是仿射变换（Affine Transformation）方程：

$${}^0\mathbf{p} = {}^0\mathbf{R}_B \cdot {}^B\mathbf{p} + {}^0\mathbf{t}_B$$

该形式存在一个明显的代数缺陷：包含一次矩阵乘法（旋转）与一次向量加法（平移）。当存在多级连续变换时，代数展开式会迅速演变为嵌套嵌套结构（如 $\mathbf{R}_1(\mathbf{R}_2\mathbf{p} + \mathbf{t}_2) + \mathbf{t}_1$ ），无法直接利用矩阵乘法的结合律进行优雅的链式相乘。

为了将三维空间中的旋转与平移统一为纯粹的单次矩阵向量乘法，引入特殊欧几里得群 $SE(3)$ 中的 4 × 4 齐次变换矩阵。

齐次坐标与 4 × 4 齐次变换矩阵的标准结构

齐次坐标表示： 在三维笛卡尔坐标向量末尾添加标量 1，扩展为四维列向量：

$$\tilde{\mathbf{p}} = \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix} \in \mathbb{R}^4$$

齐次变换矩阵结构：将 3×3 旋转矩阵 $\mathbf{R}$ 与 3×1 平移向量 $\mathbf{t}$ 拼装，并在底部补齐扩展行 $[0,0,0,1]$ ：

$$\mathbf{T} = \begin{bmatrix} \mathbf{R}_{3 \times 3} & \mathbf{t}_{3 \times 1} \\ \mathbf{0}_{1 \times 3} & 1 \end{bmatrix} = \begin{bmatrix} r_{11} & r_{12} & r_{13} & t_x \\ r_{21} & r_{22} & r_{23} & t_y \\ r_{31} & r_{32} & r_{33} & t_z \\ 0 & 0 & 0 & 1 \end{bmatrix} \in \mathbb{R}^{4 \times 4}$$

空间几何变换被简化为标准的线性矩阵乘积：

$${}^0\tilde{\mathbf{p}} = {}^0\mathbf{T}_B \cdot {}^B\tilde{\mathbf{p}} \Rightarrow \begin{bmatrix} {}^0\mathbf{p} \\ 1 \end{bmatrix} = \begin{bmatrix} {}^0\mathbf{R}_B & {}^0\mathbf{t}_B \\ \mathbf{0} & 1 \end{bmatrix} \begin{bmatrix} {}^B\mathbf{p} \\ 1 \end{bmatrix} = \begin{bmatrix} {}^0\mathbf{R}_B \cdot {}^B\mathbf{p} + {}^0\mathbf{t}_B \\ 1 \end{bmatrix}$$

计算完成后，截取前三维分量 ${}^0\tilde{\mathbf{p}}[0:3]$ 即可获得世界坐标系下的三维物理坐标。

NumPy 核心矩阵操作语法

构建 4×4 矩阵与切片赋值：

```
T = np.eye(4, dtype=np.float64) #生成 4x4 单位矩阵
T[:3, :3] = R_composite        #将前 3 行前 3 列替换为旋转矩阵
T[:3, 3] = t_vec                #将前 3 行第 4 列替换为平移向量
```

齐次坐标扩展与降维提取：

```
p_homo = np.append(p, 1.0)      #末尾拼接 1.0 转换为 4D 齐次向量[x, y, z, 1.0]
p_prime_homo = T @ p_homo       #矩阵乘法
p_prime = p_prime_homo[:3]      #切片提取前 3 维物理坐标
```

1.6.4 实验要求

1. 设定空间目标平移向量为 $\mathbf{t} = [3.0, 2.0, 2.5]^T$ 。
2. 设定三次连续外旋参数：依次绕 X 轴旋转 45.0° 、绕 Y 轴旋转 30.0° 、绕 Z 轴旋转 60.0° ，计算复合旋转矩阵 $\mathbf{R}_{composite} = \mathbf{R}_z(60^\circ)\mathbf{R}_y(30^\circ)\mathbf{R}_x(45^\circ)$ 。
3. 编写 Python 程序，将 $\mathbf{R}_{composite}$ 与 $\mathbf{t}$ 拼接构造为 4×4 齐次变换矩阵 $\mathbf{T}$ 。
4. 设定物体的几何中心点为 ${}^B\mathbf{p} = [0.0, 0.0, 0.0]^T$ ，扩展为齐次坐标后通过矩阵乘法 $\mathbf{T} @ \mathbf{p_B_homo}$ 求解变换后的世界绝对坐标。
5. （选做）调用 animator_transform_matrix.py 中的 play_direct_matrix_animation 函数，传入变换矩阵 $\mathbf{T}$ ，观察动画输出与控制台计算结果的一致性。

1.6.5 实验步骤

1. 导入 numpy 模块并编写基础单轴旋转函数 rot_x、rot_y、rot_z。
2. 构造复合旋转矩阵 R_composite 与平移向量 t_vec。

3. 使用 `T = np.eye(4)` 创建 4 阶单位阵，通过切片赋值 `T[:3, :3] = R_composite` 与 `T[:3, 3] = t_vec` 组装齐次矩阵。
4. 定义几何中心原点 `p_B = np.array([0.0, 0.0, 0.0])`，使用 `np.append(p_B, 1.0)` 扩展为齐次坐标。
5. 计算 `p_0_homo = T @ p_B_homo` 并切片提取前三维三维物理坐标。
6. （选做）调用动画模块 `play_direct_matrix_animation(T_inc=T)` 播放直变飞行动画。

1.6.6 示例代码

```
import numpy as np

#定义基础单轴旋转矩阵构造函数
def rot_x(deg):
    """构建绕 X 轴旋转矩阵 (输入参数 deg 为角度制)"""
    r = np.deg2rad(deg)
    return np.array([
        [1.0, 0.0, 0.0],
        [0.0, np.cos(r), -np.sin(r)],
        [0.0, np.sin(r), np.cos(r)]
    ], dtype=np.float64)

def rot_y(deg):
    """构建绕 Y 轴旋转矩阵 (输入参数 deg 为角度制)"""
    r = np.deg2rad(deg)
    return np.array([
        [ np.cos(r), 0.0, np.sin(r)],
        [ 0.0, 1.0, 0.0 ],
        [-np.sin(r), 0.0, np.cos(r)]
    ], dtype=np.float64)

def rot_z(deg):
    """构建绕 Z 轴旋转矩阵 (输入参数 deg 为角度制)"""
    r = np.deg2rad(deg)
    return np.array([
        [np.cos(r), -np.sin(r), 0.0],
        [np.sin(r), np.cos(r), 0.0],
```

```

        [0.0,      0.0,      1.0]
    ], dtype=np.float64)

#计算复合旋转矩阵与平移向量
#经历 X(45°)->Y(30°)->Z(60°)三次连续外旋(矩阵左乘)
Rx = rot_x(45.0)
Ry = rot_y(30.0)
Rz = rot_z(60.0)
R_composite = Rz @ Ry @ Rx

#空间平移向量
t_vec = np.array([3.0, 2.0, 2.5], dtype=np.float64)

#构造 4x4 齐次变换矩阵 T
T = np.eye(4, dtype=np.float64)
T[:3, :3] = R_composite #填充左上角 3x3 旋转子矩阵
T[:3, 3] = t_vec        #填充右上角 3x1 平移子向量

#齐次坐标变换计算(依托物体几何中心)
# 物体自身的几何中心原点坐标
p_B = np.array([0.0, 0.0, 0.0], dtype=np.float64)

#扩展为 4 维齐次坐标向量 [0, 0, 0, 1.0]
p_B_homo = np.append(p_B, 1.0)

#执行单一齐次矩阵向量积
p_0_homo = T @ p_B_homo

#提取前三维绝对空间物理坐标
p_0 = p_0_homo[:3]

#控制台输出计算结果
print(" 4x4 齐次变换矩阵 T:\n", np.round(T, 4))
print(f"\n 几何中心在自身坐标系中的坐标 p_B:      {p_B}")
print(f"扩展后的 4D 齐次坐标向量 p_B_homo:          {p_B_homo}")
print(f"矩阵相乘得到的 4D 结果向量 p_0_homo:          {np.round(p_0_homo, 4)}")

```

```
print(f'最终在世界基坐标系下的三维物理坐标 p_0: {np.round(p_0, 4)}')
```

#[选做]演示动画

try:

```
    from animator_3d import play_3d_step_animation
```

```
    play_3d_step_animation(
```

```
        T_inc=T,          #传入 4x4 齐次变换增量矩阵
```

```
        frames_per_step=60,    #插值细分帧数
```

```
        fps=30
```

```
    )
```

except ImportError:

```
    pass
```

1.6.7 实验结果

控制台输出：

```
[[ 0.433  -0.4356  0.7891  3.3   ]
```

```
 [ 0.75    0.6597 -0.0474  2.2   ]
```

```
 [-0.5     0.6124  0.6124  2.5   ]
```

```
 [ 0.      0.      0.      1.    ]]
```

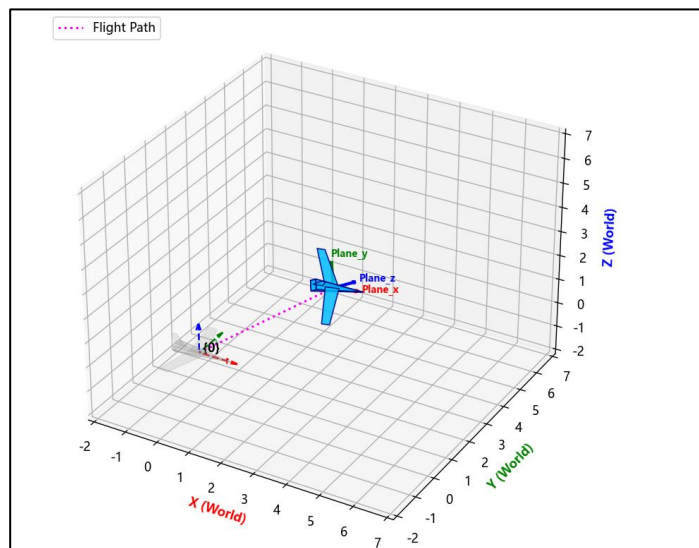
```
几何中心在自身坐标系中的坐标 p_B: [0. 0. 0.]
```

```
扩展后的 4D 齐次坐标向量 p_B_homo: [0. 0. 0. 1.]
```

```
矩阵相乘得到的 4D 结果向量 p_0_homo: [3.3 2.2 2.5 1.]
```

```
最终在世界基坐标系下的三维物理坐标 p_0: [3.3 2.2 2.5]
```

图形呈现（选做）：



1.7 实验七：多坐标系链式复合传递 ($A \rightarrow B \rightarrow C \rightarrow P$)

1.7.1 实验目的

1. 掌握空间运动学中最核心的坐标系链式乘积法则 (Kinematic Chain Multiplication Rule) 及其矩阵结合律本质。
2. 深刻理解多级串联系统 (如多关节串联机械臂、无人机多传感器与机体坐标系标定) 中位姿层级传递的正向推导机制。
3. 掌握利用 NumPy 的矩阵连续相乘运算符 @ 求解多级变换复合矩阵 0T_P 的工程实现。
4. (选做) 调用齐次矩阵动画模块中的 reference_frames 多坐标系链式接口, 动态可视化三维空间中从基准系 $\{0\}$ 经中间系 $\{B\}$ 、末端系 $\{C\}$ 直至物体目标点 P 的空间拓扑结构与飞行动画。

1.7.2 实验环境

软件环境: Python 3.8+

核心依赖包: numpy、matplotlib

工程辅助模块: animator_3d.py

1.7.3 实验原理

多坐标系串联拓扑与链式计算

在复杂的空间机器人或航天器系统中, 通常存在多个依附于不同刚体构件上的局部坐标系 (如基座坐标系 $\{A0\}$ 、肘部坐标系 $\{B\}$ 、腕部坐标系 $\{C\}$ 以及工具末端点 P)。

设相邻坐标系之间的相对位姿变换由标准的 4×4 齐次变换矩阵描述:

- 坐标系 $\{B\}$ 相对于基坐标系 $\{0\}$ 的位姿描述为： ${}^0T_B = \begin{bmatrix} {}^0R_B & {}^0t_B \\ 0 & 1 \end{bmatrix}$
- 坐标系 $\{C\}$ 相对于中间坐标系 $\{B\}$ 的位姿描述为： ${}^BT_C = \begin{bmatrix} {}^BR_C & {}^Bt_C \\ 0 & 1 \end{bmatrix}$
- 物体 P 相对于末端坐标系 $\{C\}$ 的位姿描述为： ${}^CT_P = \begin{bmatrix} {}^CR_P & {}^Ct_P \\ 0 & 1 \end{bmatrix}$

根据矩阵代数的相容性与结合律，求取末端物体 P 相对于全局基坐标系 $\{0\}$ 的综合变换矩阵时，各级矩阵遵循从基座向末端从左至右依次连乘（链式法则）：

$${}^0T_P = {}^0T_B \cdot {}^BT_C \cdot {}^CT_P$$

物体几何中心绝对物理坐标的齐次求解

由于物体的几何中心在自身局部坐标系中为原点 ${}^Pp_{origin} = [0,0,0]^T$ （对应四维齐次坐标 $\tilde{p} = [0,0,0,1]^T$ ），其在全局基准坐标系 $\{0\}$ 下的绝对三维坐标 0p 满足：

$${}^0\tilde{p} = {}^0T_P \cdot \begin{bmatrix} 0 \\ 0 \\ 0 \\ 1 \end{bmatrix} = \begin{bmatrix} {}^0R_P & {}^0t_P \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 0 \\ 0 \\ 0 \\ 1 \end{bmatrix} = \begin{bmatrix} {}^0t_P \\ 1 \end{bmatrix}$$

即物体几何中心在全局基坐标系 $\{0\}$ 下的绝对位置向量，直接等于复合齐次矩阵 0T_P 的前 3 行第 4 列平移子向量：

$${}^0p = {}^0T_P[0:3,3]$$

1.7.4 实验要求

1.已知串联空间多坐标系的相对位姿定义如下：

- 1) $0 \rightarrow B$ （基座到中间系）： $\{B\}$ 相对 $\{0\}$ 绕基准 Z_0 轴旋转 90.0° ，沿 Z_0 轴平移 ${}^0t_B = [0.0,0.0,2.0]^T$ ；
- 2) $B \rightarrow C$ （中间系到末端系）： $\{C\}$ 相对 $\{B\}$ 绕当前 X_B 轴旋转 45.0° ，沿当前 X_B 方向平移 ${}^Bt_C = [2.0,0.0,0.0]^T$ ；
- 3) $C \rightarrow P$ （末端系到目标物体）：物体 P 位于 $\{C\}$ 中，自身相对 $\{C\}$ 绕 Y_C 轴抬头 30.0° ，平移 ${}^Ct_P = [0.0,1.5,0.0]^T$ 。

2.计算任务：

- 1) 编写辅助函数 `build_T(R, t)` 构建各级 4×4 齐次变换矩阵 0T_B 、 BT_C 与 CT_P 。
 - 2) 利用 NumPy 链式相乘求解中间复合矩阵 ${}^0T_C = {}^0T_B \cdot {}^BT_C$ 以及总复合矩阵 ${}^0T_P = {}^0T_B \cdot {}^BT_C \cdot {}^CT_P$ 。
 - 3) 提取物体几何中心 P 在基准坐标系 $\{0\}$ 下的绝对三维空间物理坐标。
- 3.（选做）调用 `animator_transform_matrix.py` 驱动小飞机完成多坐标系链式飞行动画。

1.7.5 实验步骤

1. 导入 numpy 模块并编写单轴旋转函数 rot_x、rot_y、rot_z 与齐次矩阵拼接函数 build_T。
2. 分别定义并组装齐次矩阵 T_O_B、T_B_C、T_C_P。
3. 执行链式矩阵乘法：T_O_C = T_O_B @ T_B_C, T_O_P = T_O_B @ T_B_C @ T_C_P。
4. 提取绝对坐标 p_final_world = T_O_P[:3, 3] 并格式化输出。
5. （选做）构造 reference_frames=[frame_B, frame_C], 调用 play_direct_matrix_animation 展示三维动画。

1.7.6 示例代码

```
import numpy as np

#定义基础单轴旋转矩阵与齐次矩阵构造函数
def rot_x(deg):
    """构建绕 X 轴旋转矩阵 (输入参数 deg 为角度制)"""
    r = np.deg2rad(deg)
    return np.array([
        [1.0, 0.0, 0.0],
        [0.0, np.cos(r), -np.sin(r)],
        [0.0, np.sin(r), np.cos(r)]
    ], dtype=np.float64)

def rot_y(deg):
    """构建绕 Y 轴旋转矩阵 (输入参数 deg 为角度制)"""
    r = np.deg2rad(deg)
    return np.array([
        [np.cos(r), 0.0, np.sin(r)],
        [0.0, 1.0, 0.0],
        [-np.sin(r), 0.0, np.cos(r)]
    ], dtype=np.float64)

def rot_z(deg):
```


"""构建绕 Z 轴旋转矩阵 (输入参数 deg 为角度制)"""

```
r = np.deg2rad(deg)
return np.array([
    [np.cos(r), -np.sin(r), 0.0],
    [np.sin(r), np.cos(r), 0.0],
    [0.0, 0.0, 1.0]
], dtype=np.float64)
```

def build_T(R, t):

"""将 3x3 旋转矩阵与 3x1 平移向量组装为 4x4 齐次变换矩阵"""

```
T = np.eye(4, dtype=np.float64)
T[:3, :3] = R
T[:3, 3] = t
return T
```

#构造各级局部相对齐次变换矩阵

#坐标系{0}->坐标系{B}

```
R_0B = rot_z(90.0)
t_0B = np.array([0.0, 0.0, 2.0], dtype=np.float64)
T_0_B = build_T(R_0B, t_0B)
```

#坐标系{B}->坐标系{C}

```
R_BC = rot_x(45.0)
t_BC = np.array([2.0, 0.0, 0.0], dtype=np.float64)
T_B_C = build_T(R_BC, t_BC)
```

#坐标系{C}->目标物体 P

```
R_CP = rot_y(30.0)
t_CP = np.array([0.0, 1.5, 0.0], dtype=np.float64)
T_C_P = build_T(R_CP, t_CP)
```

#矩阵链式连乘求解多级复合位姿

#计算坐标系{C}在基坐标系{A}下的总位姿

```
T_0_C = T_0_B @ T_B_C
```

```
#链式连乘：计算物体 P 在基坐标系{A}下的总齐次变换矩阵
```

```
T_0_P = T_0_B @ T_B_C @ T_C_P
```

```
#提取物体几何中心在世界基坐标系{A}下的三维绝对物理坐标
```

```
p_final_world = T_0_P[:3, 3]
```

```
#控制台输出计算结果
```

```
print("坐标系 {B} 相对于 {0} 的齐次矩阵 T_A_B:\n", np.round(T_0_B, 4))
```

```
print("\n 坐标系 {C} 相对于 {B} 的齐次矩阵 T_B_C:\n", np.round(T_B_C, 4))
```

```
print("\n 坐标系 {C} 在世界坐标系 {0} 下的复合齐次矩阵 T_A_C = T_A_B @ T_B_C:\n",  
np.round(T_0_C, 4))
```

```
print("\n 物体 P 在时间坐标系 {0} 下的总齐次矩阵 T_A_P:\n", np.round(T_0_P, 4))
```

```
print(f"\n 物体几何中心 P 在世界基坐标系 {{0}} 下的最终三维坐标 [X, Y,  
Z]:\n{np.round(p_final_world, 4)}")
```

```
#[选做]调用动画模块可视化链式多坐标系运动
```

```
try:
```

```
    from animator_3d import play_3d_step_animation
```

```
    frame_B = {'T': T_0_B, 'label': '{B}'}
```

```
    frame_C = {'T': T_B_C, 'label': '{C}'}
```

```
    play_3d_step_animation(  
        reference_frames=[frame_B, frame_C], #链式派生坐标系{B}与{C}
```

```
        T_start=np.eye(4), #初始位于坐标系{C}的原点
```

```
        T_inc=T_C_P, #相对末端系{C}执行变换 T_C_P
```

```
        xlim=(-2, 4), ylim=(-1, 5), zlim=(-1, 5),
```

```
        frames_per_step=30,
```

```
        fps=30
```

```
    )
```

```
except ImportError:
```

```
    pass
```

1.7.7 实验结果

控制台输出：

坐标系 {B} 相对于 {0} 的齐次矩阵 T_{A_B} :

```
[[ 0. -1.  0.  0.]  
 [ 1.  0.  0.  0.]  
 [ 0.  0.  1.  2.]  
 [ 0.  0.  0.  1.]]
```

坐标系 {C} 相对于 {B} 的齐次矩阵 T_{B_C} :

```
[[ 1.      0.      0.      2.    ]  
 [ 0.      0.7071 -0.7071  0.    ]  
 [ 0.      0.7071  0.7071  0.    ]  
 [ 0.      0.      0.      1.    ]]
```

坐标系 {C} 在世界坐标系 {0} 下的复合齐次矩阵 $T_{A_C} = T_{A_B} @ T_{B_C}$:

```
[[ 0.     -0.7071  0.7071  0.    ]  
 [ 1.      0.     -0.      2.    ]  
 [ 0.      0.7071  0.7071  2.    ]  
 [ 0.      0.      0.      1.    ]]
```

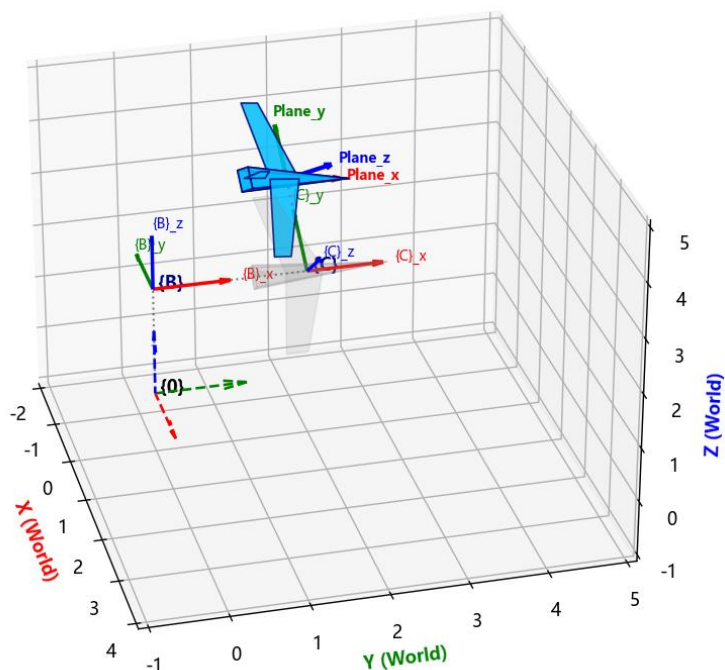
物体 P 在世界坐标系 {0} 下的总齐次矩阵 T_{A_P} :

```
[[ -0.3536 -0.7071  0.6124 -1.0607]  
 [ 0.866   0.      0.5     2.     ]  
 [ -0.3536  0.7071  0.6124  3.0607]  
 [ 0.      0.      0.      1.     ]]
```

物体几何中心 P 在世界基坐标系 {0} 下的最终三维坐标 [X, Y, Z]:

```
[-1.0607  2.      3.0607]
```

图形呈现 (选做):



1.8 实验八：多链式机构局部旋转传递与末端位姿解算

1.8.1 实验目的

1. 掌握多连杆串联机构（五自由度舵机机械臂构型）中局部正交与平行转轴的相对几何建模方法。
2. 理解在不依赖复杂 D-H 参数的前提下，直接基于相邻关节转轴的空间几何朝向与杆件偏置构建各级局部齐次变换矩阵。
3. 掌握当机构中某一个或多个局部坐标系发生自主转动时，对下游全部子坐标系空间位姿产生链式联动影响的数学本质。
4. 熟练运用 NumPy 链式矩阵乘法 (`@`)，精确求解中间关节自转后末端工具坐标系 $\{E\}$ 相对于世界基坐标系 $\{0\}$ 的综合旋转矩阵与空间位置向量。
5. （选做）调用统一三维动画模块 `animator_3d.py`，观察多级串联坐标系系统的空间拓扑结构与局部自转带动效果。

1.8.2 实验环境

1. 软件环境：Python 3.8+
2. 核心依赖包：numpy、matplotlib
3. 工程辅助模块：airplane_geometry.py、animator_3d.py

1.8.3 实验原理

本实验假设了舵机机械臂几个轴的相对位置，并建立其相关几何关系。

1. 世界系到机座系 ($0 \rightarrow A$):

机座固定于地面世界原点上方，底座旋转轴沿垂直方向 (+Z 方向)，高度偏置为 $h_0 = 1.0$:

$${}^0\mathbf{t}_A = \begin{bmatrix} 0.0 \\ 0.0 \\ 1.0 \end{bmatrix}, \quad {}^0\mathbf{R}_A = \mathbf{I}_{3 \times 3}$$

2. 机座系到肩关节系 ($A \rightarrow B$):

肩关节位于底座上方偏置 0.5 处。其俯仰旋转轴水平横置，与机座垂直转轴正交垂直。令肩关节转轴为 Z_B ，可通过绕 X_A 轴偏转 90.0° 对齐:

$${}^A\mathbf{t}_B = \begin{bmatrix} 0.0 \\ 0.0 \\ 0.5 \end{bmatrix}, \quad {}^A\mathbf{R}_B = \mathbf{R}_x(90.0^\circ)$$

3. 肩关节系到肘关节系 ($B \rightarrow C$):

大臂连杆沿当前机身延伸方向 (+ X_B) 伸展 $L_1 = 1.5$ 。由于肘关节转轴与肩关节转轴相互平行，姿态保持同向:

$${}^B\mathbf{t}_C = \begin{bmatrix} 1.5 \\ 0.0 \\ 0.0 \end{bmatrix}, \quad {}^B\mathbf{R}_C = \mathbf{I}_{3 \times 3}$$

4. 肘关节系到腕关节系 ($C \rightarrow D$):

小臂连杆沿 + X_C 延伸 $L_2 = 1.2$ 。腕部俯仰轴与肘部俯仰轴保持平行:

$${}^C\mathbf{t}_D = \begin{bmatrix} 1.2 \\ 0.0 \\ 0.0 \end{bmatrix}, \quad {}^C\mathbf{R}_D = \mathbf{I}_{3 \times 3}$$

5. 腕关节系到末端夹爪系 ($D \rightarrow E$):

末端连杆延伸 $L_3 = 0.8$ ，到达夹爪几何中心，初始姿态与 $\{D\}$ 保持对齐:

$${}^D\mathbf{t}_E = \begin{bmatrix} 0.8 \\ 0.0 \\ 0.0 \end{bmatrix}, \quad {}^D\mathbf{R}_E = \mathbf{I}_{3 \times 3}$$

(2) 多局部坐标系自转的正向运动学传递

当指定机构中的某些运动坐标系发生局部转动时，运算遵循局部右乘法则:

1. 底座系 $\{A\}$ 自转: 底座绕自身转轴 Z_A 旋转角度 $\theta_A = 45.0^\circ$:

$${}^0\mathbf{T}_A' = {}^0\mathbf{T}_A \cdot \begin{bmatrix} \mathbf{R}_z(\theta_A) & \mathbf{0} \\ \mathbf{0}^T & 1 \end{bmatrix}$$

2. 腕部系 $\{D\}$ 自转: 腕关节绕自身转轴 Z_D 旋转角度 $\theta_D = 30.0^\circ$:

$${}^C\mathbf{T}_D' = {}^C\mathbf{T}_D \cdot \begin{bmatrix} \mathbf{R}_z(\theta_D) & \mathbf{0} \\ \mathbf{0}^T & 1 \end{bmatrix}$$

根据链式传递法则，最终末端坐标系 $\{E\}$ 相对于全局基坐标系 $\{0\}$ 的总齐次变换矩阵为:

$${}^0\mathbf{T}_E = {}^0\mathbf{T}_A' \cdot {}^A\mathbf{T}_B \cdot {}^B\mathbf{T}_C \cdot {}^C\mathbf{T}_D' \cdot {}^D\mathbf{T}_E$$

截取矩阵分块即可获得末端执行器相对于世界基准的姿态与空间位置:

$${}^0\mathbf{R}_E = {}^0\mathbf{T}_E[0:3,0:3], \quad {}^0\mathbf{t}_E = {}^0\mathbf{T}_E[0:3,3]$$

1.8.4 实验要求

构建串联坐标系系统的初始状态矩阵： ${}^0\mathbf{T}_A$ 、 ${}^A\mathbf{T}_B$ 、 ${}^B\mathbf{T}_C$ 、 ${}^C\mathbf{T}_D$ 、 ${}^D\mathbf{T}_E$ 。

施加多级局部自转：

- 坐标系 $\{A\}$ 绕自身 Z_A 轴自转 45.0° ；
- 坐标系 $\{D\}$ 绕自身 Z_D 轴自转 -30.0° 。

编写程序计算：

- 旋转调整后末端工具坐标系 $\{E\}$ 相对于全局世界坐标系 $\{0\}$ 的总齐次变换矩阵 ${}^0\mathbf{T}_E$ ；
- 输出末端姿态旋转矩阵 ${}^0\mathbf{R}_E$ 与末端原点空间位置向量 ${}^0\mathbf{t}_E$ 。

（选做）调用 animator_3d.py 模块中的 play_3d_step_animation 函数，动态呈现链式系统的空间几何构型。

1.8.5 实验步骤

1. 导入 numpy 模块并编写基础单轴旋转函数 rot_x、rot_z 以及齐次矩阵拼接函数 build_T。
2. 按照各关节几何参数分别实例化基础变换矩阵。
3. 针对自转关节执行局部右乘，更新其相对齐次矩阵。
4. 使用 NumPy 的矩阵乘法运算符 @ 依次连乘求得总矩阵 T_0_E。
5. 切片提取 E 相对于实际坐标系 0 的旋转矩阵与平移矩阵并格式化打印。
6. （选做）配置参考坐标系列表并调用三维动画接口。

1.8.6 示例代码

```
import numpy as np

#基础矩阵构造工具函数
def rot_x(deg):
    """构建绕 X 轴旋转矩阵 (输入参数 deg 为角度制)"""
    r = np.deg2rad(deg)
    return np.array([
        [1.0, 0.0, 0.0],
        [0.0, np.cos(r), -np.sin(r)],
        [0.0, np.sin(r), np.cos(r)]
    ], dtype=np.float64)

def rot_z(deg):
```

"""构建绕 Z 轴旋转矩阵 (输入参数 deg 为角度制)"""

```
r = np.deg2rad(deg)
return np.array([
    [np.cos(r), -np.sin(r), 0.0],
    [np.sin(r),  np.cos(r), 0.0],
    [0.0,        0.0,      1.0]
], dtype=np.float64)
```

```
def build_T(R, t):
```

"""将 3x3 旋转矩阵与 3x1 平移向量组装为 4x4 齐次变换矩阵"""

```
T = np.eye(4, dtype=np.float64)
T[:3, :3] = R
T[:3, 3] = t
return T
```

初始各级局部相对齐次变换矩阵构建 (0 -> A -> B -> C -> D -> E)

0 -> A

```
T_0_A_init = build_T(np.eye(3), [0.0, 0.0, 1.0])
```

A -> B

```
T_A_B_init = build_T(rot_x(90.0), [0.0, 0.0, 0.5])
```

B -> C

```
T_B_C_init = build_T(np.eye(3), [1.5, 0.0, 0.0])
```

C -> D

```
T_C_D_init = build_T(np.eye(3), [1.2, 0.0, 0.0])
```

D -> E

```
T_D_E_init = build_T(np.eye(3), [0.8, 0.0, 0.0])
```

#局部坐标系自转更新与代数链式复合传递

#坐标系{A}绕自身垂直转轴 Z_A 自转 45.0°(局部右乘)

```
R_rot_A = rot_z(45.0)
```

```
T_0_A_new = T_0_A_init @ build_T(R_rot_A, [0.0, 0.0, 0.0])
```

#坐标系{D}绕自身俯仰转轴 Z_D 自转-30.0°(局部右乘)

```

R_rot_D = rot_z(-30.0)
T_C_D_new = T_C_D_init @ build_T(R_rot_D, [0.0, 0.0, 0.0])

#链式复合连乘求解末端 E 在世界基准系{0}下的总位姿
T_0_E = T_0_A_new @ T_A_B_init @ T_B_C_init @ T_C_D_new @ T_D_E_init

#提取旋转矩阵与平移位置
R_0_E = T_0_E[:3, :3]
t_0_E = T_0_E[:3, 3]

#控制台格式化输出
print("末端工具坐标系 {E} 相对于世界坐标系 {0} 的总齐次变换矩阵 T_0_E:\n", np.round(T_0_E,
4))
print("\n 末端工具坐标系 {E} 相对于世界坐标系 {0} 的最终姿态旋转矩阵 R_0_E:\n",
np.round(R_0_E, 4))
print(f"\n 末端工具原点在世界坐标系 {{0}} 下的最终物理空间位置 [X, Y, Z]:\n{np.round(t_0_E,
4)}")

```

[选做]调用动画模块动态展示多级坐标系拓扑与连续局部自转

```

try:
    from animator_3d import play_3d_step_animation

    frames = [
        {'T': T_0_A_init, 'label': '{A}'},
        {'T': T_A_B_init, 'label': '{B}'},
        {'T': T_B_C_init, 'label': '{C}'},
        {'T': T_C_D_init, 'label': '{D}'},
        {'T': T_D_E_init, 'label': '{E}'}
    ]

    rotate_frames_seq = ['A', 'D']
    rotate_R_seq = [
        [rot_z(45.0)],
        [rot_z(-30.0)]
    ]

```



```
play_3d_step_animation(  
    reference_frames=frames,  
    translation_vec_start=[0.0, 0.0, 0.0],  
    rotate_frame=rotate_frames_seq,  
    rotate_R=rotate_R_seq,  
    frame_rot_timing='before',  
    xlim=(-1, 4), ylim=(-1, 4), zlim=(0, 4),  
    frames_per_step=30,  
    fps=30  
)  
except ImportError:  
    pass
```

1.8.7 实验结果

末端工具坐标系 {E} 相对于世界坐标系 {0} 的总齐次变换矩阵 T_{0_E} :

```
[[ 0.6124  0.3536  0.7071  2.3991]  
 [ 0.6124  0.3536 -0.7071  2.3991]  
 [-0.5     0.866   0.      1.1   ]  
 [ 0.      0.      0.      1.    ]]
```

末端工具坐标系 {E} 相对于世界坐标系 {0} 的最终姿态旋转矩阵 R_{0_E} :

```
[[ 0.6124  0.3536  0.7071]  
 [ 0.6124  0.3536 -0.7071]  
 [-0.5     0.866   0.    ]]
```

末端工具原点在世界坐标系 {0} 下的最终物理空间位置 $[X, Y, Z]$:

```
[2.3991 2.3991 1.1   ]
```

1.9 实验九：六轴舵机机械臂多关节联动与夹爪控制实机验证

1.9.1 实验目的

1. 掌握上位机 Python 通过标准串口总线与 STM32 机械臂主控板进行二进制控制帧交互的

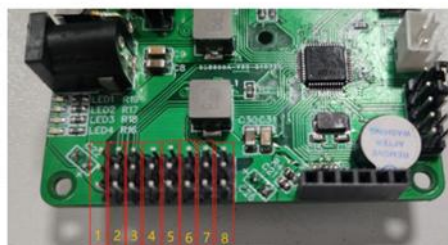
工程实现方法。

2. 理解多轴总线舵机驱动中，利用关节空间线性插值（Linear Interpolation）消除阶跃冲击、保护减速齿轮与主控电源的控制机制，并熟练调用实验直接提供的平滑驱动接口完成运动控制（插值驱动底层算法已预先封装提供，无需实验人员自行从零编写）。
3. 掌握六轴机械臂物理关节的映射关系与符号定义，依次完成单轴旋转、肘部抬升、夹爪开合以及五轴空间协同运动编排。
4. 将实验七中的链式正向运动学理论在真实物理机械臂硬件平台上进行闭环动作映射与物理姿态验证。

1.9.2 实验环境

1. **硬件环境**：六自由度总线/PWM 舵机机械臂实验台、STM32 驱动主板、USB-to-UART 串口通信线、12V/5A 直流稳压电源。
2. **软件环境**：Python 3.8+
3. **核心依赖包**：pyserial、numpy

STM32 接口	接口 1	接口 2	接口 3	接口 4	接口 5	接口 6	接口 7	接口 8
机器人关节	预留	预留	关节 6	关节 2	关节 5	关节 4	关节 3	关节 1



1.9.3 实验原理

16 字节下行总线通信协议

上位机向 STM32 机械臂控制器发送固定长度为 16 字节（Bytes）的定长二进制控制帧：

$$\text{Frame} = [0xAA, q_{1H}, q_{1L}, \dots, q_{6H}, q_{6L}, 0x01, \text{Checksum}, 0xBB]$$

- **帧头 (0xAA) 与帧尾 (0xBB)**：固定同步头尾，用于下位机状态机进行帧对齐与边界校验；
- **关节弧度数值量化**：各关节输入物理角度 θ_i （弧度制）统一放大 1000 倍，并采用大端模式（Big-Endian）编码为 16 位有符号整型数（int16）：

$$\text{val}_i = \text{int}(\theta_{i,\text{rad}} \times 1000), \quad \text{Byte}_{\text{high}} = (\text{val}_i \gg 8) \& 0xFF, \quad \text{Byte}_{\text{low}} = \text{val}_i \& 0xFF$$

- **控制模式位**：固定置为 0x01（标准关节位置伺服模式）；
- **XOR 校验和**：对 Byte 0 至 Byte 13（共 14 个字节）进行逐字节异或累加：

$$\text{Checksum} = \bigoplus_{k=0}^{13} \text{Byte}[k]$$

舵机硬件响应迅速且输出扭矩大，若直接下发大跨度的目标阶跃角度，瞬间峰值电流极易导致 STM32 主控板欠压复位（Brown-out Reset），同时也会对机械臂精密减速齿轮造成打齿冲击。

动作过渡必须经过离散时间步长 $\Delta t = \frac{T}{N}$ 的线性插值分解：

$$\mathbf{q}_k = \mathbf{q}_{start} + \frac{k}{N} \cdot (\mathbf{q}_{target} - \mathbf{q}_{start}), \quad k = 1, 2, \dots, N$$

提示：为保障硬件实验安全并让教学重心聚焦于空间位姿映射与动作编排，底层通信协议打包函数 `pack_frame` 与关节线性插值平滑驱动函数 `move_joint_interpolated` 作为工程基础设施直接提供给实验人员，不要求实验人员自行从零编写插值算法。实验人员仅需理解其抗冲击机理并在主程序中按接口规范传入姿态数组即可。

1.9.4 实验要求

基础设施复用说明：通信打包函数 `pack_frame` 与多轴线性插值驱动函数 `move_joint_interpolated` 均已在示例代码中完整给出，实验人员直接调用即可，无需重写底层插值逻辑。

通信配置要求：使用 `pyserial` 建立上位机与 STM32 的串行通信，配置波特率为 115200 bps，显式设置 `dtr=False` 与 `rts=False` 规避硬件复位卡死陷阱。

动作编排任务：在主函数中配置目标姿态数组，调用提供的插值接口，依次完成以下 4 个阶段的实机测试流程：

- **阶段 1（底座单轴旋转）：**控制关节 1 旋转 $+45.0^\circ$ ，其余关节保持 0° ，动作耗时 1.5 秒；
- **阶段 2（肘部单关节俯仰）：**控制关节 3 旋转 $+30.0^\circ$ ，其余关节维持原位，动作耗时 1.5 秒；
- **阶段 3（末端夹爪开合测试）：**控制关节 6 执行一次开合动作（张开 $+50.0^\circ$ ，停留 1.0 秒，闭合复位至 0.0° ）；
- **阶段 4（1~5 关节多轴协同联动）：**控制关节 1 至关节 5 同时旋转，到达目标姿态 $[30.0^\circ, -20.0^\circ, 35.0^\circ, -15.0^\circ, 20.0^\circ, 0.0^\circ]$ ，平稳保持 2.0 秒后，全体关节平滑复位至全零位。

1.9.5 实验步骤

1. 检查机械臂 12V 直流稳压电源供电，将 USB 通信线插入计算机。
2. 打开操作系统设备管理器，确认实际分配的串口号（如 Windows 下为 COM5，Linux 下为 `/dev/ttyUSB0`），并在主程序中修改 `SERIAL_PORT`。
3. 直接复用代码模板中提供的 `pack_frame` 与 `move_joint_interpolated` 底层函数。

4. 重点编写 main() 主程序逻辑，定义关键姿态数组，调用 move_joint_interpolated 依次触发各阶段动作。
5. 运行程序，观察机械臂硬件各舵机的运转平稳度、关节联动配合度及最终复位状态。

1.9.6 示例代码

```
import time
import serial
import numpy as np

#协议打包函数
def pack_frame(q_rad):
    """将 6 轴弧度数组打包为 16 字节控制帧"""
    tx = bytearray(16)
    tx[0] = 0xAA # 帧头
    for i in range(6):
        val = int(q_rad[i] * 1000)
        val = max(min(val, 32767), -32768) #幅值限幅
        tx[1 + i * 2] = (val >> 8) & 0xFF #高 8 位(大端模式)
        tx[2 + i * 2] = val & 0xFF # 低 8 位

    tx[13] = 0x01 #控制模式

    #计算前 14 字节 XOR 校验和
    check = 0
    for b in tx[:14]:
        check ^= b
    tx[14] = check
    tx[15] = 0xBB #帧尾
    return tx

#线性插值平滑驱动函数
def move_joint_interpolated(ser, q_curr_deg, q_target_deg, duration=1.5, steps=30):
    """
    平滑驱动机械臂从当前姿态插值过渡到目标姿态
    :param ser: 已成功打开的 serial.Serial 串口对象
    """
```

```

:param q_curr_deg: 当前姿态角度数组 (度)
:param q_target_deg: 目标姿态角度数组 (度)
:param duration: 动作过渡总时间 (秒)
:param steps: 离散插值细分步数
"""

q_start_rad = np.deg2rad(q_curr_deg)
q_target_rad = np.deg2rad(q_target_deg)
dt = duration / steps

for k in range(1, steps + 1):
    ratio = k / steps
    q_k = q_start_rad + ratio * (q_target_rad - q_start_rad)
    frame = pack_frame(q_k)
    ser.write(frame)
    ser.flush()
    time.sleep(dt)

return q_target_deg.copy()

```

#实机控制主程序

```

def main():
    SERIAL_PORT = "COM5" # 请实验人员根据实际设备端口号修改

    ser = serial.Serial()
    ser.port = SERIAL_PORT
    ser.baudrate = 115200
    ser.timeout = 1
    # 核心安全配置：禁用 DTR/RTS 规避硬件复位陷阱
    ser.dtr = False
    ser.rts = False

    try:
        ser.open()
        print(f'成功连接控制板串口: {SERIAL_PORT}')
        print("等待 STM32 底层系统就绪 (1.5 秒)...")

```

```

time.sleep(1.5)

#初始全零姿态 [J1, J2, J3, J4, J5, J6]
current_pose = np.zeros(6, dtype=np.float64)

#动作 1:控制第一个关节(底座)旋转+45.0°
pose_step1 = current_pose.copy()
pose_step1[0] = 45.0
print("\n>> 动作 1:关节 1 (底座) 偏转 +45.0°")
current_pose = move_joint_interpolated(scr, current_pose, pose_step1, duration=1.5)
time.sleep(1.0)

#动作 2:控制第三个关节(肘部小臂)旋转+30.0°
pose_step2 = current_pose.copy()
pose_step2[2] = 30.0
print("\n>> 动作 2:关节 3 (肘部小臂) 抬升 +30.0°")
current_pose = move_joint_interpolated(scr, current_pose, pose_step2, duration=1.5)
time.sleep(1.0)

#动作 3:控制最后一个关节(夹爪开合)开合一次
pose_gripper_open = current_pose.copy()
pose_gripper_open[5] = 50.0
print("\n 动作 3.1:夹爪张开 (+50.0°)")
current_pose = move_joint_interpolated(scr, current_pose, pose_gripper_open,
duration=1.0)
time.sleep(1.0)

#夹爪闭合复位至 0.0°
pose_gripper_close = current_pose.copy()
pose_gripper_close[5] = 0.0
print("动作 3.2:夹爪闭合复位 (0.0°)")
current_pose = move_joint_interpolated(scr, current_pose, pose_gripper_close,
duration=1.0)
time.sleep(1.0)

#动作 4:控制 1, 2, 3, 4, 5 号关节同时旋转到达复合目标姿态
pose_multi_joints = np.array([30.0, -20.0, 35.0, -15.0, 20.0, 0.0], dtype=np.float64)
print("\n>> 动作 4:关节 1~5 协同多轴联动 ->", pose_multi_joints[:5])

```

```

        current_pose = move_joint_interpolated(ser, current_pose, pose_multi_joints,
duration=2.0)
        time.sleep(2.0)

        #动作 5:全机平滑复位至全零初始位
        pose_home = np.zeros(6, dtype=np.float64)
        print("\n>> 动作 5:全机平滑复位至全零位")
        current_pose = move_joint_interpolated(ser, current_pose, pose_home, duration=2.0)

except Exception as e:
    print(f"硬件驱动异常: {e}")
finally:
    if ser.is_open:
        ser.close()
        print("串口物理链路已安全释放。")

if __name__ == '__main__':
    main()

```

1.9.7 实验结果

机械臂正常运作

1.10 实验十：六轴机械臂抓取、搬运实验

1.10.1 实验目的

1. 掌握利用六维关节目标数组编排机械臂连续任务的方法。
2. 理解“到达抓取位—夹爪闭合—搬运到目标位—停留—复位”的基本抓取搬运流程。
3. 掌握多关节线性插值对降低机械冲击、抑制舵机瞬时大电流和提高动作连续性的作用。
4. 能够根据实际设备串口号配置上位机程序，并安全完成机械臂实机动作验证。

1.10.2 实验环境

硬件环境：六自由度舵机机械臂、STM32 驱动主板、USB-to-UART 串口通信线、与机械臂匹配的独立稳压电源。

软件环境：Python 3.8 及以上版本。

核心依赖包：pyserial、numpy。

通信参数：程序默认使用 COM6，波特率为 115200 bps；实际运行时应根据设备管理器显示的端口修改。

1.10.3 实验原理

本实验沿用实验十的 16 字节下行控制协议。帧头为 0xAA，6 个关节角以“弧度值×1000”的 int16 大端格式写入 Byte 1 ~ Byte 12，Byte 13 为控制模式 0x01，Byte 14 为前 14 字节的 XOR 校验和，帧尾为 0xBB。

上位机以六维数组 [J1, J2, J3, J4, J5, J6] 表示机械臂目标姿态。move_joint_interpolated 函数在当前姿态和目标姿态之间进行等时间隔线性插值，使所有参与关节同步、平滑地到达目标位置。

抓取流程包含三个关键构型。第一构型 [30°, -30°, -80°, -15°, 0°, 0°] 使机械臂到达待抓取位置；第二构型保持 J1 ~ J5 不变并将 J6 调整为 -40°，完成夹爪闭合；第三构型 [0°, -10°, -60°, -15°, 0°, -40°] 在保持夹持状态的同时，将目标搬运至新的空间位置。

完成搬运并保持 5 秒后，程序将六个关节平滑插值回全零姿态。复位过程同时将 J6 从 -40° 恢复至 0°，因此会释放夹爪。

1.10.4 实验要求

检查机械臂活动范围，移除可能发生碰撞的障碍物，并确认舵机电源及公共地连接可靠。

根据设备管理器中的实际端口修改 SERIAL_PORT；程序示例默认配置为 COM6、115200 bps，并禁用 DTR/RTS。

依次完成到达抓取位、夹爪闭合、搬运三个阶段，记录各阶段机械臂姿态。

若实际机械零位或夹爪方向与示例不同，应先减小动作角度并进行单步测试，避免发生碰撞、过行程或夹爪持续堵转。

1.10.5 实验步骤

1. 检查机械臂结构、舵机接线和独立电源，在机械臂周围预留安全运动空间。
2. 连接 USB 串口线，在设备管理器中确认端口号，并修改程序中的 SERIAL_PORT。
3. 启动程序，等待 STM32 控制系统初始化完成。
4. 观察机械臂平滑移动至姿态 [30°, -30°, -80°, -15°, 0°, 0°]。
5. 观察 J6 从 0°移动至 -40°并完成夹爪闭合。
6. 观察机械臂在保持 J6=-40°的条件下移动至 [0°, -10°, -60°, -15°, 0°, -40°]，并保持 5 秒。
7. 观察六轴平滑复位至全零姿态，确认串口在程序结束后正常释放。

1.10.6 示例代码

```
import time
import serial
```



```
import numpy as np
```

```
#协议打包函数
```

```
def pack_frame(q_rad):
```

```
    """将 6 轴弧度数组打包为 16 字节控制帧"""
```

```
    tx = bytearray(16)
```

```
    tx[0] = 0xAA # 帧头
```

```
    for i in range(6):
```

```
        val = int(q_rad[i] * 1000)
```

```
        val = max(min(val, 32767), -32768) #幅值限幅
```

```
        tx[1 + i * 2] = (val >> 8) & 0xFF #高 8 位(大端模式)
```

```
        tx[2 + i * 2] = val & 0xFF # 低 8 位
```

```
    tx[13] = 0x01 #控制模式
```

```
    #计算前 14 字节 XOR 校验和
```

```
    check = 0
```

```
    for b in tx[:14]:
```

```
        check ^= b
```

```
    tx[14] = check
```

```
    tx[15] = 0xBB #帧尾
```

```
    return tx
```

```
#线性插值平滑驱动函数
```

```
def move_joint_interpolated(ser, q_curr_deg, q_target_deg, duration=1.5, steps=30):
```

```
    """
```

```
    平滑驱动机械臂从当前姿态插值过渡到目标姿态
```

```
    :param ser: 已成功打开的 serial.Serial 串口对象
```

```
    :param q_curr_deg: 当前姿态角度数组 (度)
```

```
    :param q_target_deg: 目标姿态角度数组 (度)
```

```
    :param duration: 动作过渡总时间 (秒)
```

```
    :param steps: 离散插值细分步数
```

```
    """
```

```
    q_start_rad = np.deg2rad(q_curr_deg)
```

```
    q_target_rad = np.deg2rad(q_target_deg)
```

```
    dt = duration / steps
```

```

for k in range(1, steps + 1):
    ratio = k / steps
    q_k = q_start_rad + ratio * (q_target_rad - q_start_rad)
    frame = pack_frame(q_k)
    ser.write(frame)
    ser.flush()
    time.sleep(dt)

return q_target_deg.copy()

```

#实机控制主程序

```

def main():
    SERIAL_PORT = "COM6" # 请实验人员根据实际设备端口号修改

    ser = serial.Serial()
    ser.port = SERIAL_PORT
    ser.baudrate = 115200
    ser.timeout = 1
    # 核心安全配置：禁用 DTR/RTS 规避硬件复位陷阱
    ser.dtr = False
    ser.rts = False

    try:
        ser.open()
        print(f"成功连接控制板串口: {SERIAL_PORT}")
        print("等待 STM32 底层系统就绪 (1.5 秒)...")
        time.sleep(1.5)

        #初始全零姿态 [J1, J2, J3, J4, J5, J6]
        current_pose = np.zeros(6, dtype=np.float64)

        #动作 1
        pose_multi_joints = np.array([30.0, -30.0, -80.0, -15.0, 0.0, 0.0], dtype=np.float64)

```

```

        print("\n>> 动作 1:关节 1~6 协同多轴联动 ->", pose_multi_joints[:5])
        current_pose = move_joint_interpolated(ser, current_pose, pose_multi_joints,
duration=2.0)
        time.sleep(2.0)
        #动作 2
        pose_multi_joints = np.array([30.0, -30.0, -80.0, -15.0, 0.0, -40.0], dtype=np.float64)
        print("\n>> 动作 2:关节 1~6 协同多轴联动 ->", pose_multi_joints[:5])
        current_pose = move_joint_interpolated(ser, current_pose, pose_multi_joints,
duration=2.0)
        time.sleep(2.0)
        #动作 3
        pose_multi_joints = np.array([0.0, -10.0, -60.0, -15.0, 0.0, -40.0], dtype=np.float64)
        print("\n>> 动作 3:关节 1~6 协同多轴联动 ->", pose_multi_joints[:5])
        current_pose = move_joint_interpolated(ser, current_pose, pose_multi_joints,
duration=2.0)
        time.sleep(5.0)

        #动作 4:全机平滑复位至全零初始位
        pose_home = np.zeros(6, dtype=np.float64)
        print("\n>> 动作 4:全机平滑复位至全零位")
        current_pose = move_joint_interpolated(ser, current_pose, pose_home, duration=2.0)

    except Exception as e:
        print(f"硬件驱动异常: {e}")
    finally:
        if ser.is_open:
            ser.close()
            print("串口物理链路已安全释放。")

if __name__ == '__main__':
    main()

```

1.10.7 实验结果

机械臂依次完成到达抓取位置、夹爪闭合、目标搬运、定时保持和全轴复位；动作过程连续平稳，程序结束后串口链路正常释放。

附录：

工程模块函数接口介绍

三维空间运动与多坐标系链式驱动动画模块 animator_3d.py 接口介绍

1. 原型函数展示

```
def play_3d_step_animation(
    translation_vec_inc=None,
    rotation_matrices_inc=None,
    T_inc=None,
    translation_vec_start=None,
    rotation_matrices_start=None,
    T_start=None,
    reference_frames=None,
    rotate_frame=None,
    rotate_R=None,
    frame_rot_timing='before',
    frame_mode='world',
    order='rotate_first',
    xlim=(-2, 7), ylim=(-2, 7), zlim=(-2, 7),
    frames_per_step=30, fps=30
)
```

2.形参介绍

参数名称	数据类型	默认值	物理意义与使用规范
translation_vec_inc	array_like	None	物体 P 沿参考轴发生的空间平移位移增量向量 $[t_x, t_y, t_z]$ 。若为 None 或全零则不执行平移。

参数名称	数据类型	默认值	物理意义与使用规范
rotation_matrices_inc	ndarray / list	None	物体 P 的姿态旋转增量。支持单个 3×3 旋转矩阵或连续多步旋转列表（如 [Rx, Ry, Rz]）。每个矩阵行列式必须严格满足 $\det(\mathbf{R}) = 1$ 。
T_inc	ndarray	None	4×4 齐次变换增量矩阵 ($SE(3)$ 算子)。若提供该参数，系统采用轴角测地线与空间线性插值，将旋转与平移合并为一步到位的平滑飞行动画。
translation_vec_start	array_like	None	物体 P 在运动起始时刻相对于基准坐标系（链式多坐标系中的最后一个坐标系）的原点初始位置 $[x_0, y_0, z_0]$ 。缺省时默认为 $[0.0, 0.0, 0.0]$ 。
rotation_matrices_start	ndarray / list	None	物体 P 在运动起始时刻相对于基准坐标系的初始姿态朝向（单个 3×3 矩阵或矩阵列表）。缺省时默认为 3×3 单位矩阵 $\mathbf{I}_3$ 。
T_start	ndarray	None	物体 P 在起始时刻相对于基准坐标系的 4×4 齐次位姿矩阵。用于一次性同时设定初始位置与初始姿态。
reference_frames	list[dict] / list[ndarray]	None	构建多级链式派生参考坐标系系统（如 $\{0\} \rightarrow \{A\} \rightarrow \{B\} \rightarrow \{C\}$ ）。支持字典格式 $[[\text{'t': ..., 'R': ..., 'label': '\{A\}'}]]$ 或纯 4×4 矩阵列表。
rotate_frame	str / int /	None	指定多坐标系链中需要执行原

参数名称	数据类型	默认值	物理意义与使用规范
	list		地自转的局部运动坐标系目标： 1. 单坐标系自转：传入字符串（如 'A'、'{B}'）或数值序号（如 1）； 2. 多坐标系连续自转：传入列表（如 ['A','B']），按列表索引顺序依次执行。 【安全拦截】 严禁传入 '0'、'{0}' 或 'world'！
rotate_R	ndarray / list	None	指定 rotate_frame 所选中的局部坐标系发生的自转矩阵： 1. 单坐标系自转：传入单个 3×3 矩阵或连续旋转列表 [Rx, Ry]； 2. 多坐标系连续自转：采用嵌套列表（列表套列表）组织，外层对应各坐标系，内层为该坐标系执行的连续旋转矩阵列表，例如 [[rot_z(45.0)], [rot_x(30.0), rot_y(45.0)]]。
frame_rot_timing	str	'before'	局部坐标系自转与物体 P 增量动作的时序先后关联：

参数名称	数据类型	默认值	物理意义与使用规范
			• 'before'（前置自转）：先驱动各局部坐标系自转带动下游坐标系到位，随后物体 P 相对变动后的末端坐标系执行平移/旋转增量； • 'after'（后置自转）：物体 P 先执行增量运动到达世界空间指定点位；随后局部坐标系发生自转。 【核心机制】无论 before 还是 after，自转期间物体 P 在世界坐标系中的空间绝对位置与姿态均保持不动，程序动态反求并刷新 P 相对末端系的相对坐标与关系矩阵。
frame_mode	str	'world'	旋转增量动作的参考基准轴向（仅对分步旋转有效）： • 'world'（外旋模式 / 矩阵左乘）：以当前基准参考坐标系的固定轴向为基准旋转（若有平移力臂将产生公转）； • 'body'（内旋模式 / 矩阵右乘）：以飞机自身当前的随动机身轴为基准原地纯自转。

参数名称	数据类型	默认值	物理意义与使用规范
order	str	'rotate_first'	当同时传入平移增量与旋转增量时，指定二者动作执行的先后顺序： • 'rotate_first'（标准时序）：先执行姿态旋转调姿，后执行直线空间平移； • 'translate_first'（时序颠倒对比）：先执行空间平移，后绕世界基准轴执行旋转（用于观察公转偏移）。
xlim, ylim, zlim	tuple[float, float]	(-2, 7) 等	Matplotlib 三维立体画布的各轴坐标显示范围，格式为 (min, max)。
frames_per_step	int	30	每一个独立子动作阶段（如单次平移、单轴自转）的数值插值细分帧数，数值越大过渡越细腻平滑。
fps	int	30	动画刷新帧率（Frames Per Second）。

3.使用示例

示例 1：单齐次矩阵 4×4 飞行动画

```
#构造 4x4 齐次变换矩阵 T
T_demo = build_T(rot_y(30.0) @ rot_x(45.0), [3.0, 2.0, 2.5])

play_3d_step_animation(
    T_inc=T_demo,
```



```

        frames_per_step=50,
        fps=30
    )

```

示例 2: 分步连续多轴旋转与平移

```

#先旋转 X(45°) -> Y(40°) -> Z(60°), 后沿空间平移 [2.0, 1.5, 2.0]
play_3d_step_animation(
    translation_vec_inc=[2.0, 1.5, 2.0],
    rotation_matrices_inc=[rot_x(45.0), rot_y(40.0), rot_z(60.0)],
    order='rotate_first',
    frame_mode='world'
)

```

示例 3: 多坐标系链式复合传递

```

frame_B = {'T': T_0_B, 'label': '{B}'}
frame_C = {'T': T_B_C, 'label': '{C}'}

# 物体 P 初始位于末端坐标系 {C} 原点, 相对 {C} 执行变换增量 T_C_P
play_3d_step_animation(
    reference_frames=[frame_B, frame_C],
    T_start=np.eye(4),
    T_inc=T_C_P,
    xlim=(-2, 4), ylim=(-1, 5), zlim=(-1, 5)
)

```

示例 4: 多坐标系连续局部自转与联动带动

```

# 构造多级机构初始构型: {0} -> {A} -> {B} -> {C} -> {D} -> {E}
frames = [
    {'T': T_0_A_init, 'label': '{A}'},
    {'T': T_A_B_init, 'label': '{B}'},
    {'T': T_B_C_init, 'label': '{C}'},
    {'T': T_C_D_init, 'label': '{D}'},
    {'T': T_D_E_init, 'label': '{E}'}
]

# 指定自转序列: A 坐标系自转 45° -> 不存在的 F 坐标系(自动跳过) -> D 坐标系自转 30°
play_3d_step_animation(

```

```
reference_frames=frames,
translation_vec_start=[0.0, 0.0, 0.0],
rotate_frame=['A', 'F', 'D'],
rotate_R=[
    [rot_z(45.0)],          # A 绕自身 Z_A 转 45°
    [rot_x(10.0)],          # F 坐标系不存在，输出警告后跳过
    [rot_z(30.0)]           # D 绕自身 Z_D 转 30°
],
frame_rot_timing='before',
xlim=(-1, 4), ylim=(-1, 4), zlim=(0, 4)
)
```

4.注意事项

1.依赖文件同级存放：本模块依赖 airplane_geometry.py 提取三维模型网格面片，运行时须确保两文件处于同一工作目录或 Python 系统路径下。

2.传参覆盖与互斥规则：

- 若同时传入了齐次矩阵 T_start 与分步参数 translation_vec_start 或 rotation_matrices_start，系统将直接以 T_start 为准覆盖分步参数；
- 若同时传入了齐次增量 T_inc 与分步增量参数 translation_vec_inc 或 rotation_matrices_inc，系统将优先以 T_inc 执行一体化齐次变换。

3.坐标轴色彩规范：

- 三维舞台中，世界基坐标系与随动机身坐标系正交轴颜色均遵循通用标准：X 轴（红色）、Y 轴（绿色）、Z 轴（蓝色）。

4.分步彩色残影记忆体系：

- 起始初始位置：浅灰色半透明残影；
- 绕 X 轴旋转动作：红色半透明残影；
- 绕 Y 轴旋转动作：绿色半透明残影；
- 绕 Z 轴旋转动作：蓝色半透明残影；
- 空间直线平移动作：紫色半透明残影；
- 当前实时飞机实体：深蓝描边高亮天蓝多面体。